REVISION CHEAT SHEET:

CLR / GC / MEMORY

CLR, Garbage Collection, and Memory Management in C# and .NET

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

30.1 Why This Topic Matters
30.2 What Is the CLR?
30.3 What Is IL?
30.4 JIT Compilation
30.5 Tiered Compilation
30.6 ReadyToRun and JIT
30.7 Managed Code
30.8 Managed vs Unmanaged Resources
30.9 Stack vs Heap — Avoid the Oversimplified Rule
30.10 Boxing and Memory
30.11 The Managed Heap
30.12 Allocation Example
30.13 Garbage Collection
30.14 GC Is Reachability-Based
30.15 GC Roots
30.16 Reachability Example
30.17 A Managed Memory Leak
30.18 Common Managed Leak Sources
30.19 Event Subscription Leak
30.20 Generational Garbage Collection
30.21 Generation 0
30.22 Generation 1
30.23 Generation 2
30.24 Promotion
30.25 Why Long-Lived Temporary Objects Can Hurt
30.26 GC Collection Phases
30.27 Compaction
30.28 Stop-the-World Aspect
30.29 Small Object Heap
30.30 Large Object Heap — LOH
30.31 LOH Example
30.32 Why Large Objects Are Special
30.33 LOH Pressure Example
30.34 Avoid Large Buffer Churn
30.35 Pinned Objects
30.36 Pinned Object Heap — POH
30.37 Do Not Pin Without a Reason
30.38 Finalization
30.39 Why Finalizers Cost More
30.40 Dispose vs Finalizer
30.41 SafeHandle
30.42 Weak References
30.43 ConditionalWeakTable
30.44 Workstation GC vs Server GC
30.45 Check Current GC Mode
30.46 Background GC
30.47 GC Latency Modes
30.48 No-GC Region
30.49 Do Not Call GC.Collect Routinely
30.50 Allocation Pressure
30.51 Common Allocation Sources
30.52 String Allocation Example
30.53 LINQ and Allocations
30.54 Closures and Captures
30.55 Span<T>
30.56 Span over Existing Array
30.57 ReadOnlySpan<T>
30.58 Span Restrictions
30.59 Memory<T>
30.60 stackalloc
30.61 Do Not stackalloc Huge Buffers

30.62 stackalloc Is Not Zeroed Like new[]
30.63 ArrayPool<T>
30.64 Rent May Return a Larger Array
30.65 Returned Pool Arrays May Contain Old Data
30.66 Do Not Use an Array After Returning It
30.67 ArrayPool Ownership Flow
30.68 Object Pooling
30.69 ASP.NET Core Memory Pattern — Request Scope
30.70 ASP.NET Core Memory Pattern — Bad Singleton Cache
30.71 Large Request/Response Buffering
30.72 EF Core Tracking and Memory
30.73 Do Not Retain HttpContext
30.74 Memory Leak Diagnosis Strategy
30.75 dotnet-counters
30.76 dotnet-gcdump
30.77 dotnet-dump
30.78 Memory Investigation Example
30.79 Security Considerations
30.80 Thread Safety and Memory
30.81 Common Mistake — Thinking GC Means No Leaks
30.82 Common Mistake — Forcing GC After Every Operation
30.83 Common Mistake — Huge Temporary Arrays
30.84 Common Mistake — Unbounded Cache
30.85 Common Mistake — Long-Lived DbContext
30.86 Common Mistake — Finalizer for Managed Objects
30.87 Common Mistake — Returning Pooled Array Too Early
30.88 Common Mistake — stackalloc in Large Loops
30.89 Common Mistake — Premature Allocation Optimization
30.90 Best Practices
30.91 Practical Exercise 1 — Identify the Leak
30.92 Practical Exercise 2 — Gen Promotion
30.93 Practical Exercise 3 — LOH
30.94 Practical Exercise 4 — Avoid Copy
30.95 Practical Exercise 5 — stackalloc
30.96 Practical Exercise 6 — CLR / GC / MEMORY
30.97 Practical Exercise 7 — Find a Long-Lived Root
30.98 Practical Exercise 8 — Finalizer Decision
30.99 Practical Exercise 9 — Stream Large Export
30.100 Practical Exercise 10 — Pool Ownership
30.101 Practical Exercise 11 — GC Diagnostics
30.102 Practical Exercise 12 — ASP.NET Core Cache
Topic 30 — 25 Interview Questions
1. What is the CLR?
2. What is JIT compilation?
3. What is the managed heap?
4. Do all value types live on the stack?
5. What is a GC root?
6. How does the GC decide whether an object is alive?
7. Can a managed .NET application have a memory leak?
8. What are Gen 0, Gen 1, and Gen 2?
9. What is object promotion?
10. What is a full GC?
11. What is the Large Object Heap?
12. Why can large arrays hurt performance?
13. What is pinning?
14. What is the POH?
15. What is a finalizer?
16. What does GC.SuppressFinalize do?
17. Why should you usually avoid GC.Collect?
18. What is Span<T> ?
19. What is the difference between Span<T> and Memory<T> ?
20. What is stackalloc?
21. What is ArrayPool<T> ?
22. What is the most important ArrayPool safety rule?
23. What is Server GC?
24. How do you investigate a .NET memory leak?
25. What is the most important memory-management rule for ASP.NET Core?
Topic 30 — Quick Revision
Topic 30 — Final End-to-End Flow

30.1 Why This Topic Matters

C# gives us automatic memory management, but automatic does not mean that memory stops mattering.

A production .NET application can still suffer from:

EXAMPLE / FLOW

```
Excessive allocations
Frequent garbage collections
Large Object Heap pressure
Memory fragmentation
Managed memory leaks
Long-lived object graphs
Improper caching
Finalizer overhead
Pinned-memory problems
Unbounded buffers
Large response materialization
Incorrect use of object pools
```

Understanding the runtime helps you answer questions such as:

EXAMPLE / FLOW

```
Where does a new object live?

When does an object become garbage?

Why does memory sometimes keep growing?

Why does a Gen 2 collection cost more?

Why are 100 KB arrays special?

What does pinning do?

Why can an event subscription leak memory?

When should I use Span<T>?

When should I rent an array instead of allocating one?

Why should I almost never call GC.Collect() manually?
```

30.2 What Is the CLR?

CLR means:

EXAMPLE / FLOW

```
Common Language Runtime
```

It is the runtime environment that executes managed .NET applications.

The CLR provides services including:

EXAMPLE / FLOW

```
JIT compilation
Garbage collection
Managed memory
Type safety
Exception handling
Thread/runtime services
Interop support
Metadata/type loading
```

A normal C# program follows this broad path:

EXAMPLE / FLOW

```
C# Source
   ↓
C# Compiler
   ↓
CIL / IL + Metadata
   ↓
.dll / .exe Assembly
   ↓
CLR Loads Assembly
   ↓
JIT Compilation
   ↓
Native Machine Code
   ↓
CPU Execution
```

The compiler normally produces Common Intermediate Language plus metadata. The CLR can then JIT-compile methods into native code when they are needed.

30.3 What Is IL?

Consider:

C#

```
public static int Add(
    int left,
    int right)
{
    return left + right;
}
```

The C# compiler generally does not directly produce one final machine-code version for one CPU.

It produces an intermediate representation:

EXAMPLE / FLOW

```
CIL / IL
```

along with metadata describing:

EXAMPLE / FLOW

```
Types
Methods
Fields
References
Assemblies
Attributes
```

That intermediate representation allows .NET to later produce machine code for the current runtime environment.

30.4 JIT Compilation

JIT means:

EXAMPLE / FLOW

```
Just-In-Time compilation
```

When a method is first needed, the runtime can compile its IL into native machine instructions.

Conceptually:

EXAMPLE / FLOW

```
Method IL
   ↓
JIT
   ↓
Native x64 / ARM64 instructions
   ↓
Execute
```

Later calls can use the already generated native code for that process rather than compiling the method again immediately.

30.5 Tiered Compilation

Modern .NET can use **tiered compilation**.

The basic idea is:

EXAMPLE / FLOW

```
Method first becomes hot
        ↓
Generate code quickly
        ↓
Application starts running sooner
        ↓
Runtime observes usage
        ↓
Important/hot method
        ↓
Recompile with stronger optimization
```

Current .NET supports a tiered model where an early tier favors fast code generation or precompiled code, while a later tier can produce more optimized code. Tiered compilation has been enabled by default since .NET Core 3.0.

30.6 ReadyToRun and JIT

Modern .NET can also use:

EXAMPLE / FLOW

```
ReadyToRun
```

precompiled code.

A useful mental model is:

EXAMPLE / FLOW

```
Pure JIT
    →
Compile method when needed

ReadyToRun
    →
Some native code available earlier

Tiered compilation
```

```
    →
Hot code may later be optimized further
```

Do not design business logic around assumptions about exactly which tier a method is currently using.

These are runtime implementation/performance concerns.

30.7 Managed Code

Managed code is code whose execution is managed by the CLR.

For example:

C#

```
public decimal CalculateTotal(
    IEnumerable<Gift> gifts)
{
    return gifts.Sum(
        gift =>
            gift.TotalAmount);
}
```

The runtime participates in:

EXAMPLE / FLOW

```
Allocation
Object lifetime tracking
Garbage collection
Exceptions
Type information
Execution
```

30.8 Managed vs Unmanaged Resources

Managed memory:

EXAMPLE / FLOW

```
Gift
Campaign
string
List<Gift>
byte[]
Dictionary<TKey,TValue>
```

is managed by the GC.

But a .NET application can also interact with resources outside ordinary GC-managed object memory:

EXAMPLE / FLOW

```
File handles
Database connections
Sockets
Native memory
Operating-system handles
Graphics resources
```

The GC can reclaim the **managed wrapper object**, but resources such as OS handles generally require deterministic cleanup.

That is why:

```
C#
IDisposable
```

and:

```
C#
IAsyncDisposable
```

exist.

Topic 19 covered disposal in detail.

30.9 Stack vs Heap — Avoid the Oversimplified Rule

A common beginner rule says:

```
EXAMPLE / FLOW
Value type = stack
Reference type = heap
```

That is not a reliable rule.

A better rule is:

> **Reference-type instances normally live in managed object storage, while the storage location of a value type depends on where that value is contained and how it is used.**

Example:

```
C#
public void Process()
{
    int count = 10;

    Gift gift = new();
}
```

Conceptually:

```
EXAMPLE / FLOW
Method execution state

count = 10

gift reference ──────────────┐
                             ↓
                        Managed Heap
                        ┌──────────┐
                        │ Gift     │
                        └──────────┘
```

But an `int` can also exist:

```
EXAMPLE / FLOW
Inside a class object
Inside an array
Inside another struct
Inside boxed storage
```

So:

EXAMPLE / FLOW

```
value type
```

does not universally mean:

EXAMPLE / FLOW

```
stack
```

30.10 Boxing and Memory

Recall:

C#

```
int amount = 100;

object boxed = amount;
```

The value must be represented as an object.

That introduces:

EXAMPLE / FLOW

```
Boxing
```

and normally creates managed allocation.

Repeated boxing in hot code can therefore contribute to allocation pressure.

Generic APIs help avoid many unnecessary boxing scenarios:

C#

```
List<int>
```

instead of legacy object-based containers.

30.11 The Managed Heap

The garbage collector manages a region called the:

EXAMPLE / FLOW

```
Managed Heap
```

Conceptually:

EXAMPLE / FLOW

```
Managed Heap

┌──────────────────────┐
│ Gift                 │
├──────────────────────┤
│ Campaign             │
├──────────────────────┤
│ GiftResponse         │
├──────────────────────┤
│ string               │
├──────────────────────┤
```

```
| byte[]                |
└───────────────────────┘
*
```

Managed allocation is designed to be fast. For normal heap allocation the runtime can often allocate consecutively by advancing an allocation pointer instead of searching through arbitrary free blocks.

30.12 Allocation Example

```
C#
Gift gift = new();
```

Broadly:

```
EXAMPLE / FLOW
Request memory for Gift
        ↓
GC-managed allocation area has room?
        ↓
Yes
        ↓
Advance allocation pointer
        ↓
Initialize object
        ↓
Return reference
```

This explains an important fact:

> **Allocation itself can be cheap; allocation volume and the resulting garbage-collection work are what often become expensive.**

30.13 Garbage Collection

The GC is the:

```
EXAMPLE / FLOW
Garbage Collector
```

It automatically finds managed objects that are no longer reachable and reclaims their memory.

Conceptually:

```
EXAMPLE / FLOW
Objects allocated
      ↓
Application runs
      ↓
Some objects stop being reachable
      ↓
GC occurs
      ↓
Dead objects reclaimed
      ↓
Memory reused
```

The CLR GC manages allocation and reclamation automatically.

30.14 GC Is Reachability-Based

The garbage collector does not ask:

EXAMPLE / FLOW

```
Did the source variable go out of scope?
```

The important question is:

> **Is the object reachable from a GC root?**

Example:

EXAMPLE / FLOW

```
GC Root
   ↓
Object A
   ↓
Object B
   ↓
Object C
```

All three are reachable.

But:

EXAMPLE / FLOW

```
Object D
```

with no path from any GC root is garbage.

30.15 GC Roots

Important kinds of roots include references associated with:

EXAMPLE / FLOW

```
Thread stacks / registers
Static references
GC handles
Runtime finalization structures
```

The GC begins at these roots and follows references to discover the live object graph.

30.16 Reachability Example

C#

```
public static class GiftCache
{
    public static readonly
        List<Gift> Gifts = [];
}
```

Then:

C#

```
GiftCache.Gifts.Add(
    gift);
```

Reference graph:

EXAMPLE / FLOW

```
Static Root
    ↓
GiftCache.Gifts
    ↓
List<Gift>
    ↓
Gift
    ↓
Campaign
    ↓
Other referenced objects
```

Even when a request finishes, this entire reachable graph can remain alive.

30.17 A Managed Memory Leak

A managed memory leak commonly means:

EXAMPLE / FLOW

```
Object no longer useful to application
        ↓
But still reachable
        ↓
GC correctly keeps it alive
```

Example:

C#

```
private static readonly
    List<GiftResponse>
    _history = [];
```

Every request:

C#

```
_history.Add(
    response);
```

Nothing is ever removed.

Memory keeps growing.

The GC is not broken.

The reference graph is the problem.

Microsoft's current memory-leak diagnostic guidance similarly defines a managed leak as retaining references to objects the application no longer needs.

30.18 Common Managed Leak Sources

Watch for:

EXAMPLE / FLOW

```
Static collections
Singleton caches without limits
Event subscriptions
Long-lived delegates
Timers
```

```
Background queues with no bound
Captured closures
Never-completed Tasks
EF Core tracking contexts kept alive too long
Large session state
Unbounded in-memory logs
```


30.19 Event Subscription Leak

From Topic 23:

EXAMPLE / FLOW

```
Long-Lived Publisher
        ↓
Event Delegate
        ↓
Subscriber Handler
        ↓
Subscriber Object
```

If the publisher remains reachable:

EXAMPLE / FLOW

```
subscriber remains reachable
```

unless the subscription is removed or the publisher itself dies.

30.20 Generational Garbage Collection

.NET uses a generational GC.

The main generations are:

EXAMPLE / FLOW

```
Generation 0

Generation 1

Generation 2
```

Why?

Because many applications follow this pattern:

EXAMPLE / FLOW

```
Most new objects die quickly.

Objects that survive repeatedly
are more likely to remain alive longer.
```

So the runtime can collect young objects frequently without scanning all old objects every time.

30.21 Generation 0

Generation 0 contains the youngest ordinary objects.

Examples in a Web API may include:

EXAMPLE / FLOW

```
Temporary DTOs
Short-lived strings
LINQ iterator objects
Temporary lists
Formatting objects
Request-specific helper objects
```

Most short-lived garbage is ideally reclaimed here.

Gen 0 collections are usually the most frequent.

30.22 Generation 1

Generation 1 acts partly as a buffer between:

EXAMPLE / FLOW

```
very short-lived
```

and:

EXAMPLE / FLOW

```
long-lived
```

objects.

Objects that survive a Gen 0 collection can be promoted to Gen 1.

If they continue surviving, they may move to Gen 2.

30.23 Generation 2

Generation 2 generally contains longer-lived objects.

Examples:

EXAMPLE / FLOW

```
Long-lived singleton state
Caches
Application metadata
Long-lived service objects
Objects repeatedly surviving collections
```

A Gen 2 collection is commonly called a:

EXAMPLE / FLOW

```
Full GC
```

because collecting a generation includes that generation and younger generations.

30.24 Promotion

Typical lifecycle:

EXAMPLE / FLOW

```
New Object
    ↓
Gen 0
    ↓ survives
Gen 1
```

```
    ↓ survives
Gen 2
```

Once an object is Gen 2:

EXAMPLE / FLOW

```
it remains Gen 2
```

while alive.

Objects are promoted when they survive collections.

30.25 Why Long-Lived Temporary Objects Can Hurt

Suppose a temporary object accidentally remains referenced long enough to survive several collections.

It may be promoted:

EXAMPLE / FLOW

```
Gen 0
 ↓
Gen 1
 ↓
Gen 2
```

If it becomes garbage only after reaching Gen 2, its memory may not be reclaimed until a more expensive collection considers that generation.

Therefore:

EXAMPLE / FLOW

```
Object lifetime
```

matters in addition to:

EXAMPLE / FLOW

```
Object size
```

30.26 GC Collection Phases

A simplified compacting collection includes phases such as:

EXAMPLE / FLOW

```
Mark
    ↓
Find reachable objects

Relocate
    ↓
Update references as needed

Compact
    ↓
Move survivors together
and reclaim gaps
```

Microsoft describes marking, relocation, and compaction as key GC phases.

30.27 Compaction

Before:

EXAMPLE / FLOW
```
[A][dead][B][dead][dead][C]
```

After compaction:

EXAMPLE / FLOW
```
[A][B][C][free free free]
```

Compaction helps reduce fragmentation and improves locality.

But moving objects means references to them must also be adjusted.

30.28 Stop-the-World Aspect

During certain GC work, managed application threads must be suspended.

That pause contributes to:

EXAMPLE / FLOW
```
GC latency
```

Modern GC implementations perform work concurrently/background where possible, but collection still has synchronization/pause phases.

This is one reason high allocation rates can eventually affect latency.

30.29 Small Object Heap

Most ordinary managed objects belong to the:

EXAMPLE / FLOW
```
Small Object Heap
```

or SOH.

This includes normal Gen 0, Gen 1, and Gen 2 managed objects.

30.30 Large Object Heap — LOH

Large objects are treated separately.

The default important threshold is:

EXAMPLE / FLOW
```
85,000 bytes
```

Objects at or above that size are generally allocated on the:

EXAMPLE / FLOW
```
Large Object Heap
```

LOH.

30.31 LOH Example

```C#
byte[] buffer = new byte[100_000];
```

This is large enough to normally be allocated on the LOH.

Another:

```C#
int[] values = new int[100_000];
```

roughly requires:

```EXAMPLE / FLOW
400,000 bytes
```

plus object overhead, so it is also clearly a large allocation.

30.32 Why Large Objects Are Special

Moving a 100-byte object is cheap.

Moving a 50 MB array is expensive.

Therefore LOH behavior is tuned differently.

The LOH is normally collected with Gen 2 collections and is not ordinarily compacted in the same routine way as the normal compacting heap. LOH compaction can be requested in specific scenarios.

30.33 LOH Pressure Example

Bad API design:

```C#
public byte[] GenerateExport()
{
    return new byte[
        20_000_000];
}
```

If 100 concurrent requests do this:

```EXAMPLE / FLOW
20 MB × 100
    =
~2 GB of raw arrays
```

even before considering other application memory.

The issue is not merely:

```EXAMPLE / FLOW
GC exists
```

The application is creating enormous memory pressure.

30.34 Avoid Large Buffer Churn

Instead of repeatedly:

```
C#
new byte[5_000_000]
```

for a hot operation, possible alternatives include:

```
EXAMPLE / FLOW
Streaming
ArrayPool<T>
Pipelines
Chunked processing
Reuse
```

depending on requirements.

Do not optimize blindly; measure first.

30.35 Pinned Objects

The GC normally has the freedom to move managed objects during compaction.

Sometimes native interop requires an object's address to remain stable.

That object can be:

```
EXAMPLE / FLOW
Pinned
```

A pinned object:

```
EXAMPLE / FLOW
cannot be moved
```

while pinned.

A pinned `GCHandle`, for example, prevents relocation so native code can work with a stable address.
Excessive pinning reduces GC flexibility.

30.36 Pinned Object Heap — POH

Modern .NET has a special:

```
EXAMPLE / FLOW
Pinned Object Heap
```

for certain objects deliberately allocated as pinned.

For example:

```
C#
byte[] buffer =
    GC.AllocateArray<byte>(
        1024,
        pinned:
            true);
```

Current .NET runtime metrics distinguish:

EXAMPLE / FLOW

```
gen0
gen1
gen2
loh
poh
```

and POH objects are collected with Gen 2 collections.

30.37 Do Not Pin Without a Reason

Pinning is mostly needed for scenarios involving:

EXAMPLE / FLOW

```
Native interop
Fixed unmanaged access
Special low-level buffers
```

Ordinary ASP.NET Core business logic should almost never need to think:

EXAMPLE / FLOW

```
"I should pin my Gift objects."
```

30.38 Finalization

A finalizer is a fallback mechanism for cleanup associated with unmanaged resources.

Example syntax:

C#

```
public sealed class NativeBuffer
{
    ~NativeBuffer()
    {
        // Last-resort unmanaged cleanup.
    }
}
```

A finalizer should not be your normal cleanup strategy.

Microsoft recommends preferring `SafeHandle` instead of writing custom finalizers when working with unmanaged handles because finalization is complex and error-prone.

30.39 Why Finalizers Cost More

A normal dead object can simply become reclaimable.

A finalizable object requires extra lifecycle work.

Conceptually:

EXAMPLE / FLOW

```
Object becomes unreachable
        ↓
Has finalizer?
        ↓
Yes
        ↓
Must be retained for finalization
        ↓
```

```
Finalizer eventually runs
 *      ↓
Later GC can reclaim memory
```

CLR / GC / MEMORY

This typically extends the lifetime of the object and increases GC work.

30.40 Dispose vs Finalizer

Use:

```C#
Dispose()
```

for deterministic cleanup.

Finalization is:

```EXAMPLE / FLOW
fallback protection
```

not:

```EXAMPLE / FLOW
preferred normal cleanup
```

A normal disposable design might call:

```C#
GC.SuppressFinalize(
    this);
```

after unmanaged cleanup has already been performed so the finalizer no longer needs to run.

30.41 SafeHandle

For unmanaged operating-system handles:

```C#
SafeHandle
```

is usually safer than directly owning raw native handles and writing complex finalization logic yourself.

This centralizes reliable native handle lifetime behavior.

30.42 Weak References

Normally:

```EXAMPLE / FLOW
Strong reference
    →
keeps object reachable
```

A:

```C#
WeakReference<T>
```

can refer to an object without necessarily keeping that object alive.

Conceptually:

EXAMPLE / FLOW

```
Weak reference
    ↓
Object can still be collected
if no strong roots remain
```

Weak references are specialized tools, not a general substitute for cache design.

30.43 ConditionalWeakTable

`ConditionalWeakTable<TKey,TValue>` associates extra data with object instances without causing the key to remain alive solely because it is in the table.

It is often useful for:

EXAMPLE / FLOW

```
Attaching metadata to object instances
Framework/compiler infrastructure
Specialized caches tied to object lifetime
```

Unlike a normal dictionary, the table does not keep a key alive merely because it is stored there.

30.44 Workstation GC vs Server GC

.NET provides different GC configurations.

Broadly:

EXAMPLE / FLOW

```
Workstation GC
    →
often favors interactive responsiveness
and smaller application scenarios

Server GC
    →
optimized for throughput on server workloads
using multiple GC heaps/threads
```

Server GC can provide a heap and dedicated GC thread per logical processor and perform collections across them together. It can also consume more resources, so it is not automatically best for every deployment.

30.45 Check Current GC Mode

C#

```
bool serverGc = GCSettings.IsServerGC;
```

This tells you whether the current process is using server GC.

Do not assume from:

EXAMPLE / FLOW

```
"It is an ASP.NET Core project"
```

alone without considering the actual runtime/deployment configuration.

30.46 Background GC

Modern GC can perform significant work concurrently with application execution.

The goal is to reduce long blocking pauses while still reclaiming memory.

Even with background GC:

EXAMPLE / FLOW

```
zero GC pause
```

is not guaranteed.

30.47 GC Latency Modes

.NET exposes latency modes including:

EXAMPLE / FLOW

```
Batch
Interactive
LowLatency
SustainedLowLatency
```

through:

C#

```
GCSettings.LatencyMode
```

Low-latency modes trade more conservative collection behavior and potentially larger memory use for reduced intrusive collection during sensitive periods. They are specialized runtime-tuning tools, not normal business-code settings.

30.48 No-GC Region

For highly specialized latency-sensitive code, .NET provides:

C#

```
GC.TryStartNoGCRegion(...)
```

The application requests enough memory to execute a critical region without triggering GC.

This should not appear casually in:

EXAMPLE / FLOW

```
controllers
services
repositories
```

It is an advanced optimization requiring careful capacity planning.

30.49 Do Not Call GC.Collect Routinely

Bad:

C#

```
GC.Collect();
```

after every request.

Or:


```C#
GC.Collect();

GC.WaitForPendingFinalizers();

GC.Collect();
```

because:

```EXAMPLE / FLOW
"I want to clean memory."
```

The runtime already has sophisticated heuristics for deciding when collection is beneficial.

Microsoft states that in almost all cases applications should let the GC decide when to collect; `GC.Collect` is mainly for unusual situations and testing.

30.50 Allocation Pressure

Even if every object eventually dies correctly, excessive allocation can hurt performance.

Example:

```C#
for (
    int i = 0;
    i < 1_000_000;
    i++)
{
    string value = $"Gift-{i}";
}
```

This can create huge numbers of temporary strings.

Effects may include:

```EXAMPLE / FLOW
More Gen 0 collections
More CPU spent in GC
More memory bandwidth
More promotions if objects survive
Higher latency under load
```

30.51 Common Allocation Sources

Watch hot paths involving:

```EXAMPLE / FLOW
String concatenation
LINQ materialization
Closures
Boxing
Temporary arrays
Temporary dictionaries/lists
Serialization
JSON buffers
Large DTO graphs
```

```
Regex object creation
Repeated reflection
Repeated encoding buffers
```

CLR / GC / MEMORY

This does **not** mean:

EXAMPLE / FLOW

```
never use LINQ
never allocate strings
never create DTOs
```

It means:

> Optimize allocation only where profiling shows meaningful cost.

30.52 String Allocation Example

Repeated:

C#

```
string result = "";

foreach (
    Gift gift
    in gifts)
{
    result += gift.GiftId;
}
```

can repeatedly create new strings because strings are immutable.

Better for repeated construction:

C#

```
StringBuilder builder = new();

foreach (
    Gift gift
    in gifts)
{
    builder.Append(
        gift.GiftId);
}

string result = builder.ToString();
```

30.53 LINQ and Allocations

LINQ is not automatically inefficient.

But this:

C#

```
List<Gift> result =
    gifts
        .Where(...)
        .Select(...)
        .ToList();
```

may allocate:

EXAMPLE / FLOW

```
iterator/enumerator state
result collection
projected objects
```

depending on the source and operations.

For normal application code this is often completely acceptable.

For extremely hot code:

EXAMPLE / FLOW

```
measure
```

before replacing readable LINQ with lower-level loops.

30.54 Closures and Captures

Example:

C#

```
decimal threshold = 1000m;

IEnumerable<Gift> result =
    gifts.Where(
        gift =>
            gift.TotalAmount
            >= threshold);
```

The lambda captures:

EXAMPLE / FLOW

```
threshold
```

Depending on lifetime/usage, closures can create generated objects.

Again:

EXAMPLE / FLOW

```
closure
```

is not bad.

Unnecessary closure allocation in an extremely hot path can matter.

30.55 Span<T>

`Span<T>` represents a contiguous region of memory without necessarily allocating a new array.

Example:

C#

```
Span<char> buffer = stackalloc char[64];
```

`Span<T>` is a:

EXAMPLE / FLOW

```
ref struct
```

with lifetime restrictions that prevent it from escaping into ordinary heap storage.

30.56 Span over Existing Array

C#
```
int[] values =
[
    10,
    20,
    30,
    40,
    50
];

Span<int> middle =
    values.AsSpan(
        1,
        3);
```

`middle` represents:

EXAMPLE / FLOW
```
20
30
40
```

without allocating another array containing copies of those three values.

30.57 ReadOnlySpan<T>

Use:

C#
```
ReadOnlySpan<T>
```

when the consumer should read but not modify the underlying elements through that span.

Example:

C#
```
static int Sum(
    ReadOnlySpan<int> values)
{
    int total = 0;

    foreach (
        int value
        in values)
    {
        total += value;
    }

    return total;
}
```

30.58 Span Restrictions

Because `Span<T>` is a ref struct, it cannot be used like a normal heap-storable object.

Important restrictions include scenarios such as:

EXAMPLE / FLOW

```
Boxing
Normal reference-type fields
Crossing many await/yield boundaries
```

CLR / GC / MEMORY

For async or heap-stored memory references, `Memory<T>` / `ReadOnlyMemory<T>` are usually the corresponding abstractions.

30.59 Memory<T>

Use:

C#

```
Memory<T>
```

when a memory region needs a lifetime compatible with:

EXAMPLE / FLOW

```
Async operations
Heap storage
Longer-lived ownership
```

and obtain a temporary span when performing synchronous operations:

C#

```
Span<byte> span = memory.Span;
```

30.60 stackalloc

`stackalloc` allocates temporary storage on the stack rather than the managed heap.

C#

```
Span<byte> buffer = stackalloc byte[256];
```

The memory automatically disappears when the method's stack lifetime ends.

It does not require garbage collection.

30.61 Do Not stackalloc Huge Buffers

Bad:

C#

```
Span<byte> buffer =
    stackalloc byte[
        10_000_000];
```

The stack is limited.

Too much stack allocation can cause:

EXAMPLE / FLOW

```
StackOverflowException
```

Use small, bounded sizes only.

Microsoft recommends being conservative and generally avoiding `stackalloc` inside loops.

30.62 stackalloc Is Not Zeroed Like new[]

This matters.

Normal managed array:

```
C#
byte[] buffer = new byte[100];
```

is initialized.

`stackalloc` memory should not be assumed to contain zeros unless you initialize or clear it.

```
C#
Span<byte> buffer = stackalloc byte[100];

buffer.Clear();
```

This is also a potential security concern when handling sensitive data.

30.63 ArrayPool<T>

If your application repeatedly allocates and discards temporary arrays, consider:

```
C#
ArrayPool<T>
```

Example:

```
C#
byte[] buffer =
    ArrayPool<byte>
        .Shared
        .Rent(
            4096);

try
{
    // Use buffer.
}
finally
{
    ArrayPool<byte>
        .Shared
        .Return(
            buffer);
}
```

Pooling can reduce repeated array allocations and GC pressure.

30.64 Rent May Return a Larger Array

If you request:

```
C#
Rent(
    4096)
```

the returned array is guaranteed to be at least that size.

It may be larger.

Therefore track the logical length separately.

30.65 Returned Pool Arrays May Contain Old Data

A rented buffer is not guaranteed to be clean.

If sensitive data matters:

```
C#
ArrayPool<byte>
    .Shared
    .Return(
        buffer,
        clearArray:
            true);
```

can request clearing when the pool retains the buffer.

30.66 Do Not Use an Array After Returning It

This is extremely important:

```
C#
pool.Return(
    buffer);

// Never use buffer again.
```

After returning it, ownership has been surrendered.

Another caller may receive and modify the same physical array.

Using a returned buffer can lead to:

```
EXAMPLE / FLOW
Data corruption
Data exposure
Security vulnerabilities
```

Microsoft specifically calls continuing to use a returned pooled array a high-severity security problem.

30.67 ArrayPool Ownership Flow

```
EXAMPLE / FLOW
Pool
  ↓ Rent
Your code owns buffer
  ↓
Use it
  ↓ Return
Pool owns buffer again
```

Never:

```
EXAMPLE / FLOW
Return
  ↓
Keep using
```

30.68 Object Pooling

Some expensive reusable objects can be pooled.

But pooling is not automatically an optimization.

Pooling can introduce:

EXAMPLE / FLOW

```
Complex lifetime rules
State-reset bugs
Cross-request data leakage
Contention
Memory retention
```

Only pool where:

EXAMPLE / FLOW

```
Object creation is measurably expensive
and reuse clearly helps.
```

30.69 ASP.NET Core Memory Pattern — Request Scope

Typical request objects are short-lived.

Conceptually:

EXAMPLE / FLOW

```
HTTP Request
    ↓
Request DTO
Service objects
Temporary query projections
Response DTO
Serialization buffers
    ↓
Request finishes
    ↓
Most temporary objects become unreachable
    ↓
Gen 0 / Gen 1 cleanup
```

This is the kind of lifetime generational GC handles well.

30.70 ASP.NET Core Memory Pattern — Bad Singleton Cache

C#

```
public sealed class GiftCache
{
    private readonly
        Dictionary<long, Gift>
        _cache = [];
}
```

If this service is singleton and entries are continually added with no eviction:

EXAMPLE / FLOW

```
Request 1 → +100 objects
Request 2 → +100
```

```
Request 3 → +100
```


Eventually:

EXAMPLE / FLOW

```
Cache
    ↓
Long-lived Gen 2 object graph
```

A cache should have an intentional policy:

EXAMPLE / FLOW

```
Size limit
Expiration
Eviction
Distribution
Data ownership
```

30.71 Large Request/Response Buffering

Avoid automatically materializing very large payloads.

Bad conceptual flow:

EXAMPLE / FLOW

```
50 MB file
    ↓
Read entire file into byte[]
    ↓
Create second 50 MB copy
    ↓
Serialize/transform
    ↓
Create another large buffer
```

Memory can multiply quickly under concurrent load.

Prefer streaming where supported.

30.72 EF Core Tracking and Memory

A long-lived `DbContext` can retain tracked entities.

Bad:

EXAMPLE / FLOW

```
Singleton DbContext
    ↓
Track more entities forever
```

A normal scoped unit-of-work lifetime keeps tracking state bounded.

For read-only queries:

C#

```
.AsNoTracking()
```

can reduce tracking overhead where entity modification isn't required.

30.73 Do Not Retain HttpContext

Don't store:

```C#
HttpContext
```

in static/singleton state after the request.

Beyond correctness/threading issues, retained request graphs can keep substantial request-scoped data alive.

Pass only the values actually needed.

30.74 Memory Leak Diagnosis Strategy

If production memory keeps increasing:

EXAMPLE / FLOW

```
Step 1
    ↓
Confirm memory trend

Step 2
    ↓
Observe GC/runtime metrics

Step 3
    ↓
Capture heap/dump information

Step 4
    ↓
Find dominant object types

Step 5
    ↓
Find GC roots retaining them

Step 6
    ↓
Fix reference/lifetime design
```

30.75 dotnet-counters

`dotnet-counters` can monitor runtime metrics such as:

EXAMPLE / FLOW

```
GC heap size
Gen 0 collections
Gen 1 collections
Gen 2 collections
LOH size
POH size
Allocation-related runtime metrics
```

Microsoft recommends using counters as an initial way to confirm memory-growth behavior before collecting deeper diagnostic data.

30.76 dotnet-gcdump

`dotnet-gcdump` captures GC heap information useful for understanding:

```
EXAMPLE / FLOW
Object counts
Object sizes
Object roots
Heap growth comparison
```

It can help answer:

```
EXAMPLE / FLOW
"What is consuming managed heap memory?"
```

30.77 dotnet-dump

`dotnet-dump` can capture and analyze process dumps.

Useful commands/concepts include examining:

```
EXAMPLE / FLOW
Heap statistics
Object instances
GC roots
Thread state
```

Microsoft's memory-leak guidance uses tools such as `dotnet-counters` and `dotnet-dump` together.

30.78 Memory Investigation Example

Suppose:

```
EXAMPLE / FLOW
Managed memory:
500 MB → 1 GB → 2 GB → 4 GB
```

Do not immediately write:

```
C#
GC.Collect();
```

Instead ask:

```
EXAMPLE / FLOW
Are objects actually unreachable?

Which types dominate the heap?

Are they Gen 2?

Who references them?

Is a cache unbounded?

Are event subscribers retained?

Are request objects retained?
```

If the objects remain rooted:

EXAMPLE / FLOW

```
GC.Collect
```

cannot solve the design problem.

30.79 Security Considerations

Memory management can have security consequences.

Watch for:

EXAMPLE / FLOW

```
Sensitive data retained in caches
Pooled buffers returned without clearing
Use-after-return of ArrayPool buffers
Uninitialized memory APIs
Cross-request pooled object state
Unbounded memory causing denial of service
```

For example, `GC.AllocateUninitializedArray<T>` can skip zero-initialization and Microsoft explicitly warns that reading uninitialized content can expose stale/sensitive data. It should be used only with careful low-level reasoning.

30.80 Thread Safety and Memory

The GC itself is thread-safe.

Your objects are not automatically thread-safe because they are GC-managed.

C#

```
List<Gift> gifts = [];
```

still requires correct synchronization if concurrently mutated.

Memory management:

EXAMPLE / FLOW

```
Who owns memory?
When can it be reclaimed?
```

Thread safety:

EXAMPLE / FLOW

```
Who may access/mutate state simultaneously?
```

These are different concerns.

30.81 Common Mistake — Thinking GC Means No Leaks

Wrong:

EXAMPLE / FLOW

```
GC exists
    →
memory leaks impossible
```

Correct:

EXAMPLE / FLOW

```
GC reclaims unreachable objects

Reachable-but-unneeded objects
remain alive
```

30.82 Common Mistake — Forcing GC After Every Operation

Bad:

C#

```
ProcessBatch();

GC.Collect();
```

This disrupts GC's adaptive heuristics and can promote surviving objects or create unnecessary full collection work.

30.83 Common Mistake — Huge Temporary Arrays

Bad:

C#

```
byte[] data =
    new byte[
        50_000_000];
```

inside every request when streaming could avoid full materialization.

30.84 Common Mistake — Unbounded Cache

Bad:

C#

```
cache[giftId] = gift;
```

forever.

Every cache needs:

EXAMPLE / FLOW

```
Ownership
Lifetime
Bound
Eviction
```

30.85 Common Mistake — Long-Lived DbContext

A long-lived context can retain:

EXAMPLE / FLOW

```
Tracked entities
Relationship state
Change-tracking structures
```

for much longer than intended.

30.86 Common Mistake — Finalizer for Managed Objects

Do not write:

```C#
~GiftService()
{
    _list.Clear();
}
```

Finalizers are not intended for cleaning ordinary managed collections.

The GC already manages them.

30.87 Common Mistake — Returning Pooled Array Too Early

Bad:

```C#
byte[] buffer =
    pool.Rent(
        1024);

Task writeTask =
    stream.WriteAsync(
        buffer);

pool.Return(
    buffer);

await writeTask;
```

The asynchronous operation may still be using the buffer after it has been returned.

Ownership must last until all consumers have finished.

30.88 Common Mistake — stackalloc in Large Loops

Bad:

```C#
for (
    int i = 0;
    i < 100_000;
    i++)
{
    Span<byte> buffer = stackalloc byte[4096];

    // ...
}
```

Prefer bounded/reused allocation where appropriate.

30.89 Common Mistake — Premature Allocation Optimization

Turning readable code into unsafe pooling/span code without measurement may create:

```EXAMPLE / FLOW
Security bugs
Lifetime bugs
```

```
Complexity
No meaningful speedup
```

Optimize after profiling.

30.90 Best Practices

Prefer normal managed allocation first.

Keep object lifetimes as short as naturally possible.

Avoid retaining request-specific object graphs in:

EXAMPLE / FLOW

```
Statics
Singletons
Long-lived events
Unbounded queues
Unbounded caches
```

Use `IDisposable` for deterministic non-memory resource cleanup.

Prefer `SafeHandle` over custom finalizers for native handles.

Do not routinely call:

C#

```
GC.Collect();
```

Watch large arrays and large strings.

Stream large data instead of materializing whole payloads where possible.

Use:

EXAMPLE / FLOW

```
Span<T>
ReadOnlySpan<T>
Memory<T>
ReadOnlyMemory<T>
```

where they genuinely reduce copying/allocation.

Use `ArrayPool<T>` only with strict ownership discipline.

Clear sensitive pooled data when necessary.

Measure memory with runtime diagnostics before optimizing.

30.91 Practical Exercise 1 — Identify the Leak

C#

```
public static class AuditHistory
{
    public static List<Gift>
        AllGifts = [];
}
```

Every request adds Gifts.

Question:

Why will the GC not remove old Gifts?

Answer:

EXAMPLE / FLOW

```
Static root
    ↓
List
    ↓
Gift
```

They remain reachable.

30.92 Practical Exercise 2 — Gen Promotion

Assume an object:

EXAMPLE / FLOW

```
Starts in Gen 0
Survives Gen 0 collection
Survives later Gen 1 collection
```

Typical result:

EXAMPLE / FLOW

```
Gen 0
 ↓
Gen 1
 ↓
Gen 2
```

30.93 Practical Exercise 3 — LOH

Which is more likely to enter LOH?

C#

```
new byte[1_000]
```

or:

C#

```
new byte[100_000]
```

Answer:

EXAMPLE / FLOW

```
100,000-byte array
```

because it exceeds the default 85,000-byte threshold.

30.94 Practical Exercise 4 — Avoid Copy

C#

```
int[] values =
[
    1,
    2,
    3,
    4,
    5
```

```
];

ReadOnlySpan<int> middle =
    values.AsSpan(
        1,
        3);
```

No new three-element array is required.

30.95 Practical Exercise 5 — stackalloc

```
C#
Span<char> chars = stackalloc char[32];

chars.Clear();
```

Use only for small bounded synchronous buffers.

30.96 Practical Exercise 6 — ArrayPool

```
C#
byte[] buffer =
    ArrayPool<byte>
        .Shared
        .Rent(
            8192);

try
{
    // Work with buffer.
}
finally
{
    ArrayPool<byte>
        .Shared
        .Return(
            buffer,
            clearArray:
                true);
}
```

30.97 Practical Exercise 7 — Find a Long-Lived Root

Suppose:

```
EXAMPLE / FLOW
1,000,000 GiftSummary objects
```

appear in a heap dump.

The important next question is:

```
EXAMPLE / FLOW
What GC root is retaining them?
```

not merely:

EXAMPLE / FLOW

```
How do I call GC.Collect?
```

30.98 Practical Exercise 8 — Finalizer Decision

Class owns only:

EXAMPLE / FLOW

```
List<Gift>
string
Dictionary<long,Gift>
```

Should it have a finalizer?

Answer:

EXAMPLE / FLOW

```
No.
```

These are managed objects.

30.99 Practical Exercise 9 — Stream Large Export

Instead of:

EXAMPLE / FLOW

```
Build 500 MB byte[]
then return it
```

prefer a streaming approach where the response can be produced progressively when the API/framework supports it.

30.100 Practical Exercise 10 — Pool Ownership

After:

C#

```
pool.Return(
    buffer);
```

can you continue reading `buffer`?

Answer:

EXAMPLE / FLOW

```
No.
```

Ownership has been returned to the pool.

30.101 Practical Exercise 11 — GC Diagnostics

Memory keeps increasing.

Start with:

EXAMPLE / FLOW

```
dotnet-counters
```

to confirm the trend.

Then use:


```
EXAMPLE / FLOW
dotnet-gcdump
or
dotnet-dump
```

to inspect what is retained.

30.102 Practical Exercise 12 — ASP.NET Core Cache

If you create a singleton in-memory Gift cache, define:

```
EXAMPLE / FLOW
Maximum size
Expiration
Eviction strategy
Tenant-aware key
What data may be cached
```

before calling it production-ready.

Topic 30 — 25 Interview Questions

1. What is the CLR?

The CLR, or Common Language Runtime, is the managed execution environment for .NET. It loads assemblies, manages type information, JIT-compiles IL to native code, provides garbage collection, exception handling, runtime services, and interoperability support.

A useful flow is:

```
EXAMPLE / FLOW
C# → IL → CLR → JIT → machine code
```

2. What is JIT compilation?

JIT compilation converts intermediate language into native machine instructions while the application runs.

Modern .NET can also use tiered compilation so an initially quick implementation can later be replaced by more optimized native code for hot methods.

3. What is the managed heap?

The managed heap is GC-controlled memory used for managed object allocation.

The runtime efficiently allocates objects there and later reclaims unreachable objects automatically.

4. Do all value types live on the stack?

No.

Value-type storage depends on context.

A struct can exist:

```
EXAMPLE / FLOW
As a local
Inside an object
```

```
Inside an array
Inside another struct
Inside boxed object storage
```


The simplistic rule:

EXAMPLE / FLOW

```
value type = stack
```

is incorrect.

5. What is a GC root?

A GC root is a starting point used by the garbage collector to determine object reachability.

Examples include references associated with:

EXAMPLE / FLOW

```
Thread stacks/registers
Statics
GC handles
Runtime finalization structures
```

6. How does the GC decide whether an object is alive?

It starts from GC roots and traverses references.

If an object is reachable from a root, it is considered live.

If no root path reaches it, it is garbage and can eventually be reclaimed.

7. Can a managed .NET application have a memory leak?

Yes.

If an application keeps references to objects it no longer logically needs, those objects remain reachable and cannot be reclaimed.

Typical causes include:

EXAMPLE / FLOW

```
Static collections
Unbounded caches
Event subscriptions
Long-lived delegates
Timers
Retained request graphs
```

8. What are Gen 0, Gen 1, and Gen 2?

They are GC generations used to organize objects by survival history.

EXAMPLE / FLOW

```
Gen 0 → youngest
Gen 1 → intermediate
Gen 2 → long-lived
```

Survivors are promoted toward older generations.

9. What is object promotion?

When an object survives collection, it can move to an older generation:

The idea is that repeatedly surviving objects are likely to have longer lifetimes.

10. What is a full GC?

A Gen 2 collection is commonly called a full collection because collecting Gen 2 also includes its younger generations.

11. What is the Large Object Heap?

The LOH stores large managed objects.

The default threshold is approximately:

EXAMPLE / FLOW
```
85,000 bytes
```

or greater.

LOH objects are collected with Gen 2 and are treated differently because moving large objects is expensive.

12. Why can large arrays hurt performance?

They consume large amounts of memory, enter the LOH, can increase full-GC pressure, and can contribute to fragmentation and high working-set usage.

Repeatedly allocating large buffers under concurrency can multiply memory consumption dramatically.

13. What is pinning?

Pinning prevents the GC from moving an object.

This is useful for some native interop operations requiring a stable address, but excessive pinning reduces GC compaction flexibility.

14. What is the POH?

The Pinned Object Heap is a special heap for certain deliberately pinned allocations.

Current .NET runtime metrics identify POH separately, and these objects are collected with Gen 2.

15. What is a finalizer?

A finalizer is a runtime cleanup fallback associated mainly with unmanaged resource ownership.

It should not be used for normal managed-object cleanup.

Microsoft recommends `SafeHandle` rather than custom finalizers for most unmanaged handle scenarios.

16. What does GC.SuppressFinalize do?

If deterministic cleanup already released unmanaged resources, `GC.SuppressFinalize(this)` prevents the object's finalizer from running unnecessarily.

17. Why should you usually avoid GC.Collect?

The runtime already adjusts collection timing based on allocation and memory pressure.

Manual collections can create unnecessary pauses and collection work.

Microsoft recommends letting the GC decide except for unusual specialized cases or testing.

18. What is Span<T>?

`Span<T>` is a memory-safe, allocation-friendly view over a contiguous region of memory.

It is a `ref struct`, which gives it stack/lifetime restrictions and prevents many heap-escape scenarios.

19. What is the difference between Span<T> and Memory<T>?

`Span<T>` is stack-constrained and best for synchronous processing.

`Memory<T>` can be stored and used across scenarios requiring heap-compatible lifetime, including asynchronous APIs.

You can obtain a span from `Memory<T>` when performing synchronous work.

20. What is stackalloc?

`stackalloc` allocates temporary memory on the stack.

That memory is automatically discarded when the stack lifetime ends and does not require GC.

It should be used conservatively with small bounded sizes.

21. What is ArrayPool<T>?

`ArrayPool<T>` lets code rent and return reusable arrays instead of repeatedly allocating temporary buffers.

This can reduce allocation and GC pressure in high-throughput buffer-heavy workloads.

22. What is the most important ArrayPool safety rule?

After returning an array:

```
C#
pool.Return(
    buffer);
```

you no longer own it and must not use it.

Another caller may immediately rent the same memory.

23. What is Server GC?

Server GC is a GC mode designed for server throughput and can use multiple dedicated GC threads/heaps associated with logical processors.

It can improve throughput for some workloads but also consumes more resources, so deployment characteristics matter.

24. How do you investigate a .NET memory leak?

A practical process is:

```
EXAMPLE / FLOW
Monitor memory with dotnet-counters
        ↓
Confirm persistent growth
        ↓
Collect gcdump or process dump
        ↓
Find dominant object types
```

```
        ↓
Inspect GC roots
        ↓
Locate the reference retaining objects
```

25. What is the most important memory-management rule for ASP.NET Core?

Keep request-specific objects short-lived and avoid unintentionally promoting them into application-wide lifetime.

Be especially careful with:

EXAMPLE / FLOW

```
Statics
Singleton caches
Event subscriptions
Large buffers
Tracked entities
Background queues
Request objects
```

The GC can reclaim only what becomes unreachable.

Topic 30 — Quick Revision

Concept	Meaning
CLR	.NET managed execution runtime
IL/CIL	Intermediate compiled code
JIT	Converts IL to native code
Tiered compilation	Quick code first, stronger optimization later
Managed heap	GC-managed object memory
GC root	Starting reference for reachability
Reachable object	Considered alive
Unreachable object	Eligible for collection
Gen 0	Young objects
Gen 1	Intermediate generation
Gen 2	Long-lived generation
Promotion	Survivor moves to older generation
Full GC	Gen 2 collection
LOH	Large Object Heap
LOH threshold	~85,000 bytes by default
POH	Pinned Object Heap
Pinning	Prevent object relocation
Finalizer	Unmanaged cleanup fallback
IDisposable	Deterministic resource cleanup

Concept	Meaning
Span	Stack-constrained memory view
Memory	Heap-compatible memory view
stackalloc	Temporary stack memory
ArrayPool	Reusable array buffers
dotnet-counters	Runtime metric monitoring
dotnet-gcdump	Managed heap graph data
dotnet-dump	Full process dump diagnostics

Topic 30 — Final End-to-End Flow

EXAMPLE / FLOW

```
C# Source
    ↓
Compiler
    ↓
IL + Metadata
    ↓
CLR
    ↓
JIT / ReadyToRun
    ↓
Native Code
    ↓
Application Allocates Objects
    ↓
Managed Heap
    ↓
Objects Reachable From GC Roots?
```

Yes:

EXAMPLE / FLOW

```
Keep alive
    ↓
May survive/promote
    ↓
Gen 0 → Gen 1 → Gen 2
```

No:

EXAMPLE / FLOW

```
Garbage
    ↓
GC reclaims memory
```

Special memory:

EXAMPLE / FLOW

```
Large object
    →
LOH
```

EXAMPLE / FLOW

```
Pinned special allocation
    →
POH
```

Performance question:

EXAMPLE / FLOW

```
Too many allocations?
    ↓
Measure first
    ↓
Reduce copies
Stream
Span<T>
Pool buffers
Improve lifetime/cache design
```

Leak question:

EXAMPLE / FLOW

```
Memory keeps growing?
    ↓
Find retained object
    ↓
Find GC root
    ↓
Fix reference lifetime
```

REVISION CHEAT SHEET:

SOLID / DESIGN

SOLID and Core Design Principles in C# and .NET

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

31.1 Why Design Principles Matter

31.2 SOLID Overview

31.3 Cohesion

31.4 Coupling

31.5 Design Goal

Part 1 — Single Responsibility Principle

31.6 What Is SRP?

31.7 SRP Is Not “One Method Per Class”

31.8 Bad SRP Example

31.9 Better SRP Design

31.10 SRP Fundraising Example

31.11 Constructor Dependency Smell

31.12 When Not to Split Further

Part 2 — Open/Closed Principle

31.13 What Is OCP?

31.14 Bad OCP Example

31.15 OCP with Strategy

31.16 OCP Does Not Mean “Never Modify Code”

31.17 Do Not Abstract Before You Know the Variation

Part 3 — Liskov Substitution Principle

31.18 What Is LSP?

31.19 LSP Is About Behavior

31.20 Contract Components

31.21 Preconditions

31.22 Postconditions

31.23 Exceptions and LSP

31.24 Classic Shape Problem

31.25 LSP Design Lesson

Part 4 — Interface Segregation Principle

31.26 What Is ISP?

31.27 Bad Interface

31.28 Better Segregated Interfaces

31.29 ISP Does Not Mean One-Method Interfaces Everywhere

31.30 ISP and Capabilities

Part 5 — Dependency Inversion Principle

31.31 What Is DIP?

31.32 Bad Dependency Direction

31.33 Inverted Dependency

31.34 Dependency Injection vs Dependency Inversion

Dependency Inversion

Dependency Injection

31.35 Inversion of Control

31.36 ASP.NET Core DI Example

31.37 Explicit Dependencies

Part 6 — Separation of Concerns

31.38 What Is Separation of Concerns?

31.39 Fundraising Layer Example

31.40 Bad Controller

31.41 Better Controller

Part 7 — Encapsulation and Information Hiding

31.42 Encapsulation

Better

31.43 Information Hiding

Part 8 — DRY

31.44 What Is DRY?

31.45 Bad Duplication

31.46 Better

31.47 DRY Does Not Mean “Never Repeat Code”

Part 9 — KISS

31.48 What Is KISS?

31.49 Simple Does Not Mean Primitive

Part 10 — YAGNI

31.50 What Is YAGNI?

31.51 YAGNI and OCP Balance

Part 11 — Composition over Inheritance

31.52 Why Prefer Composition Often?

31.53 Why Composition Is Flexible

31.54 When Inheritance Is Good

Part 12 — Law of Demeter

31.55 Law of Demeter

31.56 Why Deep Navigation Is Fragile

Part 13 — Tell, Don't Ask

31.57 Tell, Don't Ask

Part 14 — Principle of Least Knowledge and Explicit APIs

Part 15 — SOLID Applied Together

31.58 Improved Design

31.59 How Principles Apply

31.60 SOLID and Testing

Testable

31.61 Do Not Create Interfaces Only for Mocking

31.62 SOLID and Performance

31.63 SOLID and Memory

31.64 SOLID and Thread Safety

31.65 SOLID and Security

31.66 Service Locator Anti-Pattern

31.67 Static Dependency Anti-Pattern

31.68 God Class

31.69 Anemic Over-Splitting

31.70 Common Mistake — SRP Means One Method

31.71 Common Mistake — OCP Means Never Edit Existing Code

31.72 Common Mistake — LSP Is Just Type Compatibility

31.73 Common Mistake — ISP Means Tiny Interfaces Everywhere

31.74 Common Mistake — DIP Means “Use DI Container”

31.75 Common Mistake — DRY Everything

31.76 Common Mistake — Premature OCP

31.77 Common Mistake — Inheritance for Code Reuse

31.78 Common Mistake — Repository for Every Object Automatically

31.79 Common Mistake — Business Rules in Controller

31.80 Common Mistake — Business Rules in Infrastructure

31.81 Common Mistake — Too Many Interfaces

31.82 Best Practices

31.83 Practical Exercise 1 — SRP

31.84 Practical Exercise 2 — OCP

31.85 Practical Exercise 3 — LSP

31.86 Practical Exercise 4 — ISP

31.87 Practical Exercise 5 — DIP

31.88 Practical Exercise 6 — DRY

31.89 Practical Exercise 7 — Encapsulation

31.90 Practical Exercise 8 — Composition

31.91 Practical Exercise 9 — Explicit Dependencies

31.92 Practical Exercise 10 — KISS

31.93 Practical Exercise 11 — Multi-Tenant Security

31.94 Practical Exercise 12 — Refactor Large Service

Topic 31 — 25 Interview Questions

1. What is SOLID?
2. What is the Single Responsibility Principle?
3. How can constructor dependencies reveal an SRP problem?
4. What is the Open/Closed Principle?
5. Does OCP mean existing code must never change?
6. What is the Liskov Substitution Principle?
7. What kinds of behavior matter for LSP?
8. Why is the Rectangle/Square example associated with LSP?
9. What is Interface Segregation?
10. Does ISP require single-method interfaces?
11. What is Dependency Inversion?
12. What is the difference between DIP and dependency injection?
13. What is inversion of control?
14. Why is constructor injection usually preferred?
15. What is separation of concerns?
16. What is encapsulation?
17. What is DRY?
18. What is KISS?
19. What is YAGNI?
20. How do OCP and YAGNI coexist?
21. Why is composition often preferred over inheritance?
22. What is the Law of Demeter?
23. Can SOLID hurt a project?
24. How does SOLID improve testing?

25. How would you apply SOLID to the Fundraising Platform?
Topic 31 — Quick Revision
Topic 31 — Complete Fundraising Architecture Example
Controller
Domain Gift
Repository Abstraction
Application Service
Infrastructure Repository
Dependency Registration
Final Architecture Flow
Topics 30 + 31 — Combined Mental Model
Final Revision Flow

SOLID / DESIGN

31.1 Why Design Principles Matter

C# allows you to write this:

```C#
public sealed class GiftService
{
    public void ProcessGift(
        Gift gift)
    {
        ValidateGift(
            gift);

        SaveGift(
            gift);

        ChargePayment(
            gift);

        GenerateReceipt(
            gift);

        SendEmail(
            gift);

        WriteAuditLog(
            gift);

        UpdateCampaignTotals(
            gift);

        PublishMessage(
            gift);
    }
}
```

The compiler may be satisfied.

But design questions remain:

EXAMPLE / FLOW

```
How many reasons does GiftService have to change?

How do we test it?

How do we replace SMTP?

How do we change payment providers?

How do we add a new receipt mechanism?

What happens if auditing changes?
```

```
Can persistence change without rewriting business logic?

Can another developer understand the dependencies?
```

SOLID / DESIGN

SOLID and related design principles help answer these questions.

Microsoft's current .NET architecture guidance identifies SOLID as a foundational set of principles for internal application design and dependency decoupling.

31.2 SOLID Overview

SOLID means:

EXAMPLE / FLOW

```
S
Single Responsibility Principle

O
Open/Closed Principle

L
Liskov Substitution Principle

I
Interface Segregation Principle

D
Dependency Inversion Principle
```

A useful mental model:

EXAMPLE / FLOW

```
S
    →
Give a class one cohesive responsibility

O
    →
Create stable extension points

L
    →
Implementations must preserve the contract

I
    →
Do not force clients to depend on members they do not need

D
    →
High-level policy should depend on abstractions,
not low-level implementation details
```

31.3 Cohesion

Cohesion describes how closely related the responsibilities inside a component are.

High cohesion:

EXAMPLE / FLOW

```
GiftValidator
    ↓
Gift validation rules
```

SOLID / DESIGN

Low cohesion:

EXAMPLE / FLOW

```
GiftManager
    ↓
Validate
SQL
Email
PDF
Payment
CSV
Authentication
Caching
Logging
```

High cohesion usually improves:

EXAMPLE / FLOW

```
Understandability
Testability
Maintainability
Change isolation
```

31.4 Coupling

Coupling describes how strongly components depend on each other's details.

Tight coupling:

C#

```
public sealed class GiftService
{
    private readonly
        SqlGiftRepository
        _repository = new();

    private readonly
        SmtpReceiptSender
        _sender = new();
}
```

The service directly chooses:

EXAMPLE / FLOW

```
SQL implementation
SMTP implementation
Construction
Lifetime
```

Looser coupling:

C#

```
public sealed class GiftService
{
    private readonly
```

```
        IGiftRepository
        _repository;

    private readonly
        IReceiptSender
        _receiptSender;

    public GiftService(
        IGiftRepository repository,
        IReceiptSender receiptSender)
    {
        _repository = repository;

        _receiptSender = receiptSender;
    }
}
```

SOLID / DESIGN

Now the service knows:

EXAMPLE / FLOW

```
contracts
```

instead of:

EXAMPLE / FLOW

```
infrastructure implementations
```

31.5 Design Goal

A healthy design usually seeks:

EXAMPLE / FLOW

```
High cohesion internally
+
Low unnecessary coupling externally
+
Explicit dependencies
+
Strong contracts
```

Part 1 — Single Responsibility Principle

31.6 What Is SRP?

SRP says:

> **A class or module should have one cohesive responsibility and therefore one primary reason to change.**

Microsoft's architecture guidance expresses SRP as an object having one responsibility and one reason to change.

31.7 SRP Is Not “One Method Per Class”

Bad interpretation:

EXAMPLE / FLOW

```
One class
    →
one method
```

SOLID / DESIGN

That would create meaningless fragmentation.

A class may have many methods if they all support the same responsibility.

Example:

C#

```
public sealed class GiftValidator
{
    public void ValidateAmount(
        Gift gift)
    {
    }

    public void ValidateCampaign(
        Gift gift)
    {
    }

    public void ValidateAllocations(
        Gift gift)
    {
    }
}
```

All methods belong to:

EXAMPLE / FLOW

```
Gift validation
```

31.8 Bad SRP Example

C#

```
public sealed class GiftService
{
    public bool Validate(
        Gift gift)
    {
        // Validation.
        return true;
    }

    public void Save(
        Gift gift)
    {
        // SQL.
    }

    public void SendReceipt(
        Gift gift)
    {
        // SMTP.
    }

    public byte[] CreatePdf(
        Gift gift)
    {
```

```
        // PDF.
        return [];
    }

    public void WriteAudit(
        Gift gift)
    {
        // Logging.
    }
}
```

SOLID / DESIGN

Reasons to change include:

EXAMPLE / FLOW

```
Gift validation changes
Database changes
Email provider changes
PDF library changes
Audit requirements change
```

That is many axes of change.

31.9 Better SRP Design

EXAMPLE / FLOW

```
GiftValidator
    →
Validation rules

GiftRepository
    →
Persistence

ReceiptSender
    →
Receipt delivery

ReceiptDocumentGenerator
    →
PDF/document generation

AuditWriter
    →
Audit recording

GiftApplicationService
    →
Orchestrates the use case
```

The orchestrator can coordinate several dependencies without owning each dependency's implementation responsibility.

31.10 SRP Fundraising Example

C#

```
public sealed class GiftApplicationService
{
    private readonly
        IGiftValidator
        _validator;
```

```
    private readonly
        IGiftRepository
        _repository;

    private readonly
        IReceiptSender
        _receiptSender;

    public GiftApplicationService(
        IGiftValidator validator,
        IGiftRepository repository,
        IReceiptSender receiptSender)
    {
        _validator = validator;

        _repository = repository;

        _receiptSender = receiptSender;
    }

    public async Task ProcessAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        _validator.Validate(
            gift);

        await _repository.SaveAsync(
            gift,
            cancellationToken);

        await _receiptSender.SendAsync(
            gift,
            cancellationToken);
    }
}
```

SOLID / DESIGN

Responsibility:

EXAMPLE / FLOW

```
Orchestrate the Process Gift use case
```

Not:

EXAMPLE / FLOW

```
Implement SQL
Implement SMTP
Implement validation
```

31.11 Constructor Dependency Smell

Suppose:

C#

```
public GiftService(
    IRepository repository,
    ILogger logger,
    IEmailSender email,
    IPdfGenerator pdf,
    IPaymentGateway payment,
    ICache cache,
    IAuditWriter audit,
```

```
    IMapper mapper,
    IMessageBus bus,
    IClock clock,
    IFileStore fileStore)
```

SOLID / DESIGN

The number itself is not automatically wrong.

But it is a strong question:

EXAMPLE / FLOW

```
Does this class own too many responsibilities?
```

Microsoft's DI guidance explicitly says many constructor dependencies can indicate an SRP violation and suggests splitting responsibilities.

31.12 When Not to Split Further

Bad overdesign:

EXAMPLE / FLOW

```
GiftAmountReader
GiftAmountChecker
GiftAmountComparer
GiftAmountValidationInvoker
GiftAmountValidationCoordinator
```

for one simple rule.

SRP does not mean:

EXAMPLE / FLOW

```
maximum number of classes
```

It means:

EXAMPLE / FLOW

```
cohesive responsibilities
```

Part 2 — Open/Closed Principle

31.13 What Is OCP?

OCP says:

> **Software entities should be open for extension but closed for modification.**

Meaning:

EXAMPLE / FLOW

```
New supported behavior
    ↓
ideally add new implementation

instead of

editing a central class every time
```

The principle is about creating stable variation points, often through polymorphism, composition, configuration, delegates, or strategies.

SOLID / DESIGN

31.14 Bad OCP Example

C#

```
public decimal CalculateFee(
    Gift gift)
{
    if (
        gift.PaymentMethod
        ==
        PaymentMethod.CreditCard)
    {
        return
            gift.TotalAmount
            *
            0.03m;
    }

    if (
        gift.PaymentMethod
        ==
        PaymentMethod.Ach)
    {
        return
            1.00m;
    }

    if (
        gift.PaymentMethod
        ==
        PaymentMethod.Check)
    {
        return
            0m;
    }

    throw new
        NotSupportedException();
}
```

Now add:

EXAMPLE / FLOW

```
PayPal
Apple Pay
Wire Transfer
Crypto
```

Every new type requires modifying the same method.

31.15 OCP with Strategy

C#

```
public interface
    IPaymentFeeCalculator
{
    bool CanHandle(
        PaymentMethod method);
```

```
    decimal Calculate(
        Gift gift);
}
```

SOLID / DESIGN

Credit card:

C#

```
public sealed class
    CreditCardFeeCalculator
    :
    IPaymentFeeCalculator
{
    public bool CanHandle(
        PaymentMethod method)
        =>
        method
        == PaymentMethod.CreditCard;

    public decimal Calculate(
        Gift gift)
        =>
        gift.TotalAmount
        *
        0.03m;
}
```

ACH:

C#

```
public sealed class
    AchFeeCalculator
    :
    IPaymentFeeCalculator
{
    public bool CanHandle(
        PaymentMethod method)
        =>
        method
        == PaymentMethod.Ach;

    public decimal Calculate(
        Gift gift)
        =>
        1m;
}
```

New payment type:

EXAMPLE / FLOW

```
add implementation
```

instead of modifying the existing calculators.

31.16 OCP Does Not Mean “Never Modify Code”

Requirements change.

Bug fixes happen.

OCP does not literally prohibit modification.

The goal is:

Common variation should happen through intended extension points rather than repeatedly editing stable central logic.

31.17 Do Not Abstract Before You Know the Variation

Overengineering:

EXAMPLE / FLOW

```
One receipt format exists

but create:
IReceiptStrategyFactory
IReceiptStrategyResolver
IReceiptPipelineProvider
IReceiptContextBuilder
```

because:

EXAMPLE / FLOW

```
"Maybe someday..."
```

That violates another principle:

EXAMPLE / FLOW

```
YAGNI
```

Create extension points when variation is real or reasonably imminent.

Part 3 — Liskov Substitution Principle

31.18 What Is LSP?

LSP says:

An implementation of a subtype or abstraction must be safely usable wherever the base contract is expected.

This is stronger than:

EXAMPLE / FLOW

```
Compiler allows assignment.
```

A subtype can compile correctly and still violate behavioral expectations.

Microsoft summarizes LSP as requiring a subtype to be safely usable wherever the parent type is expected.

31.19 LSP Is About Behavior

Suppose:

C#

```
public interface IReceiptSender
{
    Task SendAsync(
```

```
        Gift gift,
        CancellationToken cancellationToken);
}
```

Implementation A:

```C#
public sealed class
    EmailReceiptSender
    :
    IReceiptSender
{
    public Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        // Send.
        return Task.CompletedTask;
    }
}
```

Implementation B:

```C#
public sealed class
    FakeReceiptSender
    :
    IReceiptSender
{
    public Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        throw new
            NotSupportedException();
    }
}
```

If callers reasonably expect:

```
EXAMPLE / FLOW
Every IReceiptSender can send receipts
```

then the second implementation violates the contract.

31.20 Contract Components

LSP asks implementations to respect things such as:

```
EXAMPLE / FLOW
Preconditions
Postconditions
Invariants
Expected exceptions
State changes
Return semantics
Side-effect expectations
```

31.21 Preconditions

Base contract:

```
EXAMPLE / FLOW
Accept any non-null Gift.
```

Derived implementation:

```
EXAMPLE / FLOW
Accept only Gift.Amount > $10,000.
```

The derived type has strengthened the requirement.

That may violate substitutability.

31.22 Postconditions

Suppose contract says:

```
EXAMPLE / FLOW
On successful SaveAsync,
Gift has been persisted.
```

An implementation that silently ignores the save violates that promise.

31.23 Exceptions and LSP

Suppose callers understand:

```
EXAMPLE / FLOW
duplicate external reference
    →
DuplicateGiftException
```

but one implementation unexpectedly throws:

```
EXAMPLE / FLOW
NotSupportedException
```

for ordinary valid usage.

That can violate behavioral substitutability.

The point is not:

```
EXAMPLE / FLOW
all implementations throw identical exceptions
```

but:

```
EXAMPLE / FLOW
implementation honors the abstraction's intended contract
```

31.24 Classic Shape Problem

Imagine:

```C#
class Rectangle
{
    public virtual int Width
    {
        get;
        set;
    }

    public virtual int Height
    {
        get;
        set;
    }
}
```

Then:

```C#
class Square :
    Rectangle
```

forces width and height to remain equal.

Code written for `Rectangle` may expect:

```C#
rectangle.Width = 5;

rectangle.Height = 10;
```

and assume:

EXAMPLE / FLOW

```
Width = 5
Height = 10
```

A `Square` cannot preserve both its own invariant and that Rectangle behavior.

Inheritance may therefore be the wrong relationship.

31.25 LSP Design Lesson

Ask:

> **Does every implementation truly satisfy the semantic contract?**

Not merely:

EXAMPLE / FLOW

```
Can I make the compiler accept it?
```

Part 4 — Interface Segregation Principle

31.26 What Is ISP?

ISP says:

Clients should not be forced to depend on operations they do not need.

SOLID / DESIGN

Large “everything” interfaces cause unnecessary coupling.

31.27 Bad Interface

C#

```
public interface IGiftManager
{
    Gift Get(
        long id);

    void Save(
        Gift gift);

    void Delete(
        long id);

    void SendReceipt(
        long id);

    byte[] ExportPdf(
        long id);

    void RebuildSearchIndex();

    void ClearCache();

    void RunAnnualReport();
}
```

A read-only endpoint needing:

EXAMPLE / FLOW

```
Get
```

now depends conceptually on an interface containing:

EXAMPLE / FLOW

```
Delete
Email
PDF
Cache
Reporting
```

31.28 Better Segregated Interfaces

C#

```
public interface IGiftReader
{
    Task<Gift?> FindAsync(
        long giftId,
        CancellationToken cancellationToken);
}
```

```C#
public interface IGiftWriter
{
    Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

```C#
public interface IReceiptSender
{
    Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

```C#
public interface IGiftExporter
{
    Task<Stream> ExportAsync(
        long giftId,
        CancellationToken cancellationToken);
}
```

Each consumer asks for only what it needs.

31.29 ISP Does Not Mean One-Method Interfaces Everywhere

Bad interpretation:

```EXAMPLE / FLOW
Every method
    →
new interface
```

Interfaces should represent cohesive roles.

Example:

```C#
public interface IGiftRepository
{
    Task<Gift?> FindAsync(...);

    Task AddAsync(...);

    Task SaveChangesAsync(...);
}
```

may be perfectly cohesive.

31.30 ISP and Capabilities

A useful approach is thinking in terms of roles:

```EXAMPLE / FLOW
Readable
```

```
Writable
*
Exportable

Notifiable
```

SOLID / DESIGN

rather than:

EXAMPLE / FLOW

```
One gigantic manager abstraction
```

Part 5 — Dependency Inversion Principle

31.31 What Is DIP?

DIP says broadly:

> **High-level policy should not depend directly on low-level implementation details. Both should depend on abstractions appropriate to the policy boundary.**

Microsoft's architectural guidance describes dependency direction as moving toward abstractions rather than implementation details.

31.32 Bad Dependency Direction

EXAMPLE / FLOW

```
GiftApplicationService
        ↓
SqlConnection
        ↓
SQL Server
```

Code:

C#

```
public sealed class
    GiftApplicationService
{
    public async Task ProcessAsync(
        Gift gift)
    {
        using SqlConnection connection =
            new(
                "...");

        // SQL.
    }
}
```

The high-level application use case knows:

EXAMPLE / FLOW

```
SQL provider
Connection construction
Persistence commands
```

31.33 Inverted Dependency

SOLID / DESIGN

Define:

```
C#
public interface IGiftRepository
{
    Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

High-level:

```
C#
public sealed class
    GiftApplicationService
{
    private readonly
        IGiftRepository
        _repository;

    public GiftApplicationService(
        IGiftRepository repository)
    {
        _repository = repository;
    }
}
```

Infrastructure:

```
C#
public sealed class
    SqlGiftRepository
    :
    IGiftRepository
{
    public Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        // EF Core / SQL implementation.
        return Task.CompletedTask;
    }
}
```

Compile-time dependency:

```
EXAMPLE / FLOW
Application
    ↓
IGiftRepository abstraction

Infrastructure
    ↓
implements IGiftRepository
```

31.34 Dependency Injection vs Dependency Inversion

These are related but different.

Dependency Inversion

SOLID / DESIGN

EXAMPLE / FLOW

```
Design principle
```

It answers:

EXAMPLE / FLOW

```
What should depend on what?
```

Dependency Injection

EXAMPLE / FLOW

```
Construction/composition technique
```

It answers:

EXAMPLE / FLOW

```
How does an object receive dependencies?
```

Example:

C#

```
public GiftService(
    IGiftRepository repository)
```

31.35 Inversion of Control

IoC is broader:

EXAMPLE / FLOW

```
Control of creation/execution
is delegated to framework/container
```

ASP.NET Core DI is one form of IoC.

31.36 ASP.NET Core DI Example

C#

```
builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();

builder.Services
    .AddScoped<
        IReceiptSender,
        EmailReceiptSender>();

builder.Services
    .AddScoped<
        GiftApplicationService>();
```

Then:

```C#
public GiftController(
    GiftApplicationService service)
{
    _service = service;
}
```

The container constructs the object graph.

ASP.NET Core's DI guidance recommends constructor injection, avoiding direct instantiation of dependent classes, and keeping services small and testable.

31.37 Explicit Dependencies

Bad:

```C#
public sealed class GiftService
{
    public Task ProcessAsync()
    {
        GiftRepository repo =
            ServiceLocator.Get<
                GiftRepository>();

        // ...
    }
}
```

The constructor suggests:

```EXAMPLE / FLOW
GiftService needs nothing
```

but runtime reality says:

```EXAMPLE / FLOW
GiftService secretly needs GiftRepository
```

Better:

```C#
public GiftService(
    IGiftRepository repository)
```

Now dependency requirements are visible.

Microsoft's architectural guidance calls this the Explicit Dependencies Principle.

Part 6 — Separation of Concerns

31.38 What Is Separation of Concerns?

SoC says:

> ⓘ Different kinds of work should be separated into appropriate components/layers.

For example:

EXAMPLE / FLOW

```
API
    →
HTTP concern

Application layer
    →
Use-case orchestration

Domain
    →
Business rules

Infrastructure
    →
Database / external systems
```

Microsoft explicitly identifies separation of concerns as a key architectural principle and a motivation for layering.

31.39 Fundraising Layer Example

EXAMPLE / FLOW

```
API
    ↓
GiftController
```

Responsibility:

EXAMPLE / FLOW

```
HTTP input/output
Status codes
Request binding
```

EXAMPLE / FLOW

```
Application
    ↓
ProcessGiftHandler
```

Responsibility:

EXAMPLE / FLOW

```
Orchestrate use case
```

EXAMPLE / FLOW

```
Domain
    ↓
Gift
GiftAllocation
```

Responsibility:

EXAMPLE / FLOW

```
Business invariants
```

EXAMPLE / FLOW

```
Infrastructure
    ↓
GiftRepository
EmailReceiptSender
```

Responsibility:

EXAMPLE / FLOW

```
Technical implementation
```

31.40 Bad Controller

C#

```
[HttpPost]
public async Task<IActionResult>
    Create(
        CreateGiftRequest request)
{
    if (
        request.Amount <= 0)
    {
        return BadRequest();
    }

    // Create SqlConnection.
    // Insert Gift SQL.
    // Call payment API.
    // Generate receipt.
    // Send SMTP.
    // Update cache.
    // Publish message.

    return Ok();
}
```

This controller owns:

EXAMPLE / FLOW

```
HTTP
Business logic
SQL
Integration
Messaging
```

31.41 Better Controller

C#

```
[HttpPost]
public async Task<
    ActionResult<GiftResponse>>
    Create(
        CreateGiftRequest request,
        CancellationToken cancellationToken)
{
    GiftResponse response =
        await _service.CreateAsync(
            request,
            cancellationToken);
```

```
    return CreatedAtAction(
        nameof(Get),
        new
        {
            giftId =
                response.GiftId
        },
        response);
}
```

Controller responsibility stays focused.

Part 7 — Encapsulation and Information Hiding

31.42 Encapsulation

Encapsulation means protecting internal state behind well-defined behavior.

Bad:

C#

```
public sealed class Gift
{
    public decimal Amount;

    public List<GiftAllocation>
        Allocations = [];
}
```

Anyone can create invalid state.

Better

C#

```
public sealed class Gift
{
    private readonly
        List<GiftAllocation>
        _allocations = [];

    public decimal Amount
    {
        get;
        private set;
    }

    public IReadOnlyList<GiftAllocation>
        Allocations
        =>
        _allocations;

    public void AddAllocation(
        GiftAllocation allocation)
    {
        if (
            allocation.Amount <= 0)
        {
            throw new
                ArgumentOutOfRangeException(
                    nameof(allocation));
```

```
        }

        decimal current =
            _allocations.Sum(
                x =>
                    x.Amount);

        if (
            current
            +
            allocation.Amount
            >
            Amount)
        {
            throw new
                InvalidOperationException(
                    "Allocation exceeds gift amount.");
        }

        _allocations.Add(
            allocation);
    }
}
```

Outside code cannot bypass the invariant through normal API usage.

Microsoft's architecture guidance connects encapsulation with insulating internal implementation and reducing coupling.

31.43 Information Hiding

Encapsulation is not merely:

```
EXAMPLE / FLOW
make fields private
```

It is also:

```
EXAMPLE / FLOW
Hide implementation decisions
that callers should not depend on.
```

Example:

```
C#
public interface IGiftRepository
{
    Task<Gift?> FindAsync(
        long giftId,
        CancellationToken cancellationToken);
}
```

Consumer does not need to know whether storage is:

```
EXAMPLE / FLOW
EF Core
Dapper
SQL
In-memory test implementation
```

Part 8 — DRY

SOLID / DESIGN

31.44 What Is DRY?

DRY means:

EXAMPLE / FLOW

```
Don't Repeat Yourself
```

More precisely:

> **A business rule or knowledge concept should not have multiple independent authoritative implementations.**

Microsoft's architecture guidance describes duplicated business behavior as dangerous because future requirement changes can leave copies inconsistent.

31.45 Bad Duplication

Controller:

C#

```
if (
    request.Amount <= 0
    ||
    request.Amount
    >
    1_000_000m)
{
}
```

Service:

C#

```
if (
    amount <= 0
    ||
    amount
    >
    1_000_000m)
{
}
```

Background import:

C#

```
if (
    gift.Amount <= 0
    ||
    gift.Amount
    >
    1_000_000m)
{
}
```

Now change maximum to:

EXAMPLE / FLOW

```
2,000,000
```

SOLID / DESIGN

Someone updates two places but misses one.

31.46 Better

C#

```
public sealed class GiftAmount
{
    public const decimal
        Maximum = 2_000_000m;

    public static void Validate(
        decimal amount)
    {
        if (
            amount
            is <= 0m
            or
            >
            Maximum)
        {
            throw new
                ArgumentOutOfRangeException(
                    nameof(amount));
        }
    }
}
```

One authoritative rule.

31.47 DRY Does Not Mean “Never Repeat Code”

Two blocks that happen to look similar may represent different business concepts.

Prematurely merging them can create artificial coupling.

Ask:

EXAMPLE / FLOW

```
Are these copies of the same knowledge?
```

not merely:

EXAMPLE / FLOW

```
Do these five lines look similar?
```

Part 9 — KISS

31.48 What Is KISS?

KISS means:

EXAMPLE / FLOW

```
Keep It Simple
```

A design should be as simple as possible while still correctly satisfying known requirements.

Example:

Need two receipt implementations?

SOLID / DESIGN

Simple:

```
C#
IReceiptSender
```

plus:

```
EXAMPLE / FLOW
EmailReceiptSender
PrintReceiptSender
```

Potentially excessive:

```
EXAMPLE / FLOW
ReceiptAbstractFactory
ReceiptFactoryProvider
ReceiptFactoryResolver
ReceiptStrategyRegistry
ReceiptPipelineBuilder
ReceiptExecutionContextFactory
```

unless those abstractions solve real complexity.

31.49 Simple Does Not Mean Primitive

A simple architecture can still have:

```
EXAMPLE / FLOW
Controllers
Application services
Domain objects
Repositories
Interfaces
DI
```

if those elements correspond to real concerns.

KISS means:

```
EXAMPLE / FLOW
No unnecessary complexity
```

not:

```
EXAMPLE / FLOW
Everything in Program.cs
```

Part 10 — YAGNI

31.50 What Is YAGNI?

YAGNI means:

Do not implement speculative flexibility without a real requirement.

Example:

Current requirement:

```
EXAMPLE / FLOW
Only SQL Server persistence.
```

You probably do not need to immediately build support for:

```
EXAMPLE / FLOW
MongoDB
Cosmos DB
Redis primary persistence
PostgreSQL
Flat files
Oracle
```

unless architecture requirements actually demand it.

31.51 YAGNI and OCP Balance

OCP says:

```
EXAMPLE / FLOW
Design extension points for real variation.
```

YAGNI says:

```
EXAMPLE / FLOW
Do not invent speculative variation.
```

Healthy balance:

```
EXAMPLE / FLOW
Known changing dimension?
    →
abstract intentionally

Unknown hypothetical future?
    →
keep design simple
```

Part 11 — Composition over Inheritance

31.52 Why Prefer Composition Often?

Inheritance creates:

```
EXAMPLE / FLOW
is-a
```

relationships.

Composition creates:

EXAMPLE / FLOW

```
has-a / uses-a
```

relationships.

Example inheritance:

C#

```
class GiftService :
    EmailService
```

This probably makes little conceptual sense.

Better:

C#

```
class GiftService
{
    private readonly
        IEmailSender
        _emailSender;
}
```

Now:

EXAMPLE / FLOW

```
GiftService
uses EmailSender
```

31.53 Why Composition Is Flexible

Inheritance binds behavior into a type hierarchy.

Composition allows behavior to be replaced independently:

EXAMPLE / FLOW

```
GiftService
    ↓
IReceiptSender
    ├─ EmailReceiptSender
    ├─ PrintReceiptSender
    └─ TestReceiptSender
```

No GiftService inheritance hierarchy is required.

31.54 When Inheritance Is Good

Inheritance still fits when:

EXAMPLE / FLOW

```
True substitutable is-a relationship exists
Base contract is stable
LSP can be honored
Shared polymorphic behavior is meaningful
```

Do not interpret:

EXAMPLE / FLOW

```
Composition over inheritance
```

SOLID / DESIGN

as:

EXAMPLE / FLOW

```
Inheritance is forbidden
```

Part 12 — Law of Demeter

31.55 Law of Demeter

A practical version says:

> **An object should collaborate mainly with its direct dependencies rather than navigating deeply through another object's internals.**

Problem:

C#

```
string code =
    gift
        .Campaign
        .Owner
        .Department
        .Address
        .Country
        .Code;
```

The caller knows a huge amount about the internal object graph.

31.56 Why Deep Navigation Is Fragile

If structure changes:

EXAMPLE / FLOW

```
Campaign.Owner
    →
Campaign.Team.Owner
```

many callers break.

Alternative:

C#

```
string GetCampaignCountryCode()
```

or better:

EXAMPLE / FLOW

```
appropriate domain/query abstraction
```

depending on the context.

Do not mechanically create forwarding methods for every property. The principle is about reducing unnecessary structural knowledge.

Part 13 — Tell, Don't Ask

SOLID / DESIGN

31.57 Tell, Don't Ask

Weak domain behavior:

```C#
if (
    gift.Status
    ==
    GiftStatus.Pending)
{
    gift.Status = GiftStatus.Completed;
}
```

Caller asks for internal state, decides rule, mutates state.

Better:

```C#
gift.Complete();
```

Inside:

```C#
public void Complete()
{
    if (
        Status
        !=
        GiftStatus.Pending)
    {
        throw new
            InvalidOperationException();
    }

    Status = GiftStatus.Completed;
}
```

The object owns its state transition rule.

Part 14 — Principle of Least Knowledge and Explicit APIs

A public API should expose:

```EXAMPLE / FLOW
what callers need
```

not:

```EXAMPLE / FLOW
everything implementation knows
```

Instead of:

```C#
public DbContext Context
{
```

```
    get;
}
```

SOLID / DESIGN

from your domain service, expose the business operation.

This reduces accidental coupling.

Part 15 — SOLID Applied Together

Consider a receipt workflow.

Bad:

C#

```
public sealed class GiftService
{
    public async Task CompleteAsync(
        Gift gift)
    {
        if (
            gift.Amount <= 0)
        {
            throw new Exception();
        }

        // EF SQL logic

        if (
            gift.ReceiptType == "Email")
        {
            // SMTP implementation
        }
        else if (
            gift.ReceiptType == "Print")
        {
            // PDF implementation
        }

        // audit implementation
    }
}
```

Problems:

EXAMPLE / FLOW

```
SRP
    →
validation + persistence + receipt + audit

OCP
    →
new receipt type modifies GiftService

ISP
    →
likely manager abstractions become broad

DIP
    →
depends directly on implementation details

LSP
    →
```

```
future polymorphic receipt implementations
may not honor consistent contract
```

31.58 Improved Design

C#

```
public interface IGiftRepository
{
    Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

C#

```
public interface IReceiptSender
{
    bool CanHandle(
        ReceiptType type);

    Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

C#

```
public interface IAuditWriter
{
    Task WriteGiftCompletedAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Orchestrator:

C#

```
public sealed class
    CompleteGiftService
{
    private readonly
        IGiftRepository
        _repository;

    private readonly
        IReadOnlyList<IReceiptSender>
        _receiptSenders;

    private readonly
        IAuditWriter
        _auditWriter;

    public CompleteGiftService(
        IGiftRepository repository,
        IEnumerable<IReceiptSender>
            receiptSenders,
        IAuditWriter auditWriter)
    {
        _repository = repository;

        _receiptSenders = receiptSenders.ToList();
```

```
        _auditWriter = auditWriter;
    }

    public async Task CompleteAsync(
        Gift gift,
        ReceiptType receiptType,
        CancellationToken cancellationToken)
    {
        gift.Complete();

        await _repository.SaveAsync(
            gift,
            cancellationToken);

        IReceiptSender sender =
            _receiptSenders
                .Single(
                    sender =>
                        sender.CanHandle(
                            receiptType));

        await sender.SendAsync(
            gift,
            cancellationToken);

        await _auditWriter
            .WriteGiftCompletedAsync(
                gift,
                cancellationToken);
    }
}
```

SOLID / DESIGN

31.59 How Principles Apply

SRP:

EXAMPLE / FLOW

```
CompleteGiftService
    →
orchestrates completion
```

OCP:

EXAMPLE / FLOW

```
New ReceiptSender implementation
    →
can extend behavior
```

LSP:

EXAMPLE / FLOW

```
Every IReceiptSender
must satisfy receipt contract
```

ISP:

EXAMPLE / FLOW

```
Receipt sender doesn't also expose
repository/report/cache methods
```

DIP:

EXAMPLE / FLOW

```
Application depends on abstractions
not SMTP/EF implementation classes
```

SOLID / DESIGN

31.60 SOLID and Testing

Tightly coupled:

C#

```
public sealed class GiftService
{
    private readonly
        SqlGiftRepository
        _repository = new();

    private readonly
        EmailReceiptSender
        _sender = new();
}
```

Unit test may require real infrastructure.

Testable

C#

```
public GiftService(
    IGiftRepository repository,
    IReceiptSender sender)
```

Test:

C#

```
IGiftRepository repository = new FakeGiftRepository();

IReceiptSender sender = new FakeReceiptSender();

GiftService service =
    new(
        repository,
        sender);
```

Now the test controls dependencies.

31.61 Do Not Create Interfaces Only for Mocking

An interface is most valuable when it expresses:

EXAMPLE / FLOW

```
A meaningful abstraction
A replaceable boundary
A role/capability
```

Bad:

EXAMPLE / FLOW

```
IGiftDto
IGiftMapperHelper
IGiftNameFormatterInternal
```

for every trivial class solely because:

EXAMPLE / FLOW

```
"SOLID says use interfaces."
```

SOLID does not say every class needs an interface.

31.62 SOLID and Performance

More abstraction can introduce:

EXAMPLE / FLOW

```
More objects
More indirection
More virtual/interface calls
More allocations in poorly designed pipelines
```

But in ordinary ASP.NET Core applications, architecture mistakes such as:

EXAMPLE / FLOW

```
bad SQL
N+1 queries
unbounded allocations
network latency
serialization
```

normally dominate a simple interface call.

Do not sacrifice maintainability for hypothetical nanoseconds without measurement.

31.63 SOLID and Memory

Overengineering can create excessively large object graphs:

EXAMPLE / FLOW

```
Controller
  ↓
Service
  ↓
Coordinator
  ↓
Factory
  ↓
Resolver
  ↓
Provider
  ↓
Adapter
  ↓
Wrapper
```

Each dependency may be small, but complexity itself has cost.

Prefer meaningful abstractions.

31.64 SOLID and Thread Safety

SOLID / DESIGN

Dependency inversion does not make an implementation thread-safe.

Example:

```
C#
public interface IGiftCache
{
}
```

implemented by:

```
C#
SingletonGiftCache
```

with a mutable:

```
C#
Dictionary<long, Gift>
```

Still needs concurrency-safe design.

Abstraction:

```
EXAMPLE / FLOW
does not remove runtime behavior requirements
```

31.65 SOLID and Security

Bad abstraction can hide authorization.

Example:

```
C#
public interface IGiftRepository
{
    Task<Gift?> FindAsync(
        long giftId);
}
```

In a multi-tenant application, a caller may accidentally request another tenant's Gift.

Potentially stronger contract:

```
C#
Task<Gift?> FindAsync(
    long tenantId,
    long giftId,
    CancellationToken cancellationToken);
```

or enforce tenant scope at an appropriate architectural boundary.

Design principles should make security requirements:

```
EXAMPLE / FLOW
explicit
```

not hidden.

31.66 Service Locator Anti-Pattern

Bad:

```C#
public sealed class GiftService
{
    public void Process()
    {
        IGiftRepository repo =
            ServiceLocator
                .Resolve<
                    IGiftRepository>();
    }
}
```

Dependencies are hidden.

Constructor injection:

```C#
public GiftService(
    IGiftRepository repository)
```

makes them visible.

31.67 Static Dependency Anti-Pattern

Bad:

```C#
public static class GiftHelpers
{
    public static EmailSender
        Email = new();
}
```

Now tests and consumers share global mutable state.

Static utility methods can be fine when:

```EXAMPLE / FLOW
Pure
Stateless
Dependency-free
```

but not as a substitute for proper service lifetime/composition.

31.68 God Class

Signs:

```EXAMPLE / FLOW
2,000+ lines
20 dependencies
dozens of unrelated methods
database + messaging + validation + reporting
changes for almost every feature
```

A God class often violates:

EXAMPLE / FLOW

```
SRP
SoC
DIP
ISP
```

Refactor by identifying cohesive responsibilities.

31.69 Anemic Over-Splitting

Opposite problem:

EXAMPLE / FLOW

```
GiftAmountGetter
GiftAmountSetter
GiftAmountRuleProvider
GiftAmountCheckExecutor
GiftAmountComparisonStrategy
```

for:

EXAMPLE / FLOW

```
if amount <= 0
```

That creates accidental complexity.

Good design is not measured by:

EXAMPLE / FLOW

```
number of interfaces
```

31.70 Common Mistake — SRP Means One Method

False.

SRP concerns reasons to change and responsibility cohesion.

31.71 Common Mistake — OCP Means Never Edit Existing Code

False.

It means known variation should preferably be handled through stable extension mechanisms rather than repeatedly modifying central logic.

31.72 Common Mistake — LSP Is Just Type Compatibility

False.

This compiles:

C#

```
IReceiptSender sender = new BrokenReceiptSender();
```

but if the implementation violates the behavioral contract, substitution is not valid.

31.73 Common Mistake — ISP Means Tiny Interfaces Everywhere

SOLID / DESIGN

False.

An interface can contain several methods when they represent one cohesive role.

31.74 Common Mistake — DIP Means “Use DI Container”

False.

A container is a mechanism.

The principle concerns dependency direction.

You can follow DIP manually without a container.

You can also use a DI container while still designing terrible abstractions.

31.75 Common Mistake — DRY Everything

Two identical-looking lines may represent different concepts.

Do not merge unrelated business rules just to eliminate visual duplication.

31.76 Common Mistake — Premature OCP

Do not create strategy systems for variation that does not exist.

Apply YAGNI.

31.77 Common Mistake — Inheritance for Code Reuse

Bad reason:

EXAMPLE / FLOW

```
"I want these three methods,
so I will inherit."
```

Inheritance is a semantic relationship, not merely a code-copy reduction mechanism.

Prefer composition if:

EXAMPLE / FLOW

```
uses-a
```

describes the relationship better than:

EXAMPLE / FLOW

```
is-a
```

31.78 Common Mistake — Repository for Every Object Automatically

Architecture principles do not require:

EXAMPLE / FLOW

```
Repository for every table
Service for every repository
Interface for every service
```

SOLID / DESIGN

Abstractions should serve actual architectural boundaries.

31.79 Common Mistake — Business Rules in Controller

Controllers should generally coordinate HTTP concerns, not become the authoritative home of domain rules.

If:

EXAMPLE / FLOW

```
Gift amount must be positive
```

is a business invariant, put that rule where all entry points honor it.

31.80 Common Mistake — Business Rules in Infrastructure

Bad:

EXAMPLE / FLOW

```
GiftRepository.Save()
    ↓
quietly decides
whether Gift can complete
```

Persistence infrastructure should not unexpectedly own core business policy.

31.81 Common Mistake — Too Many Interfaces

If:

EXAMPLE / FLOW

```
interface only has exactly one implementation
no likely variation
no architectural boundary
no role abstraction
```

ask whether the interface adds real value.

31.82 Best Practices

Use SRP to keep classes cohesive.

Use OCP where variation is real.

Use LSP to reason about behavior, not merely inheritance syntax.

Use ISP to expose focused client-facing roles.

Use DIP to point compile-time dependencies toward abstractions.

Use constructor injection to make dependencies explicit.

Keep domain logic separate from infrastructure concerns.

Use encapsulation to protect invariants.

Use DRY for duplicated knowledge, not merely duplicated text.

Use KISS to resist needless architecture.

Use YAGNI to resist hypothetical requirements.

SOLID / DESIGN

Prefer composition when inheritance does not represent a genuine substitutable relationship.

Design abstractions around behavior/capabilities.

Do not let SOLID become an excuse for creating dozens of meaningless layers.

31.83 Practical Exercise 1 — SRP

Bad:

C#

```
public sealed class GiftManager
{
    public void Validate()
    {
    }

    public void Save()
    {
    }

    public void Email()
    {
    }

    public void Export()
    {
    }
}
```

Refactor conceptually into:

EXAMPLE / FLOW

```
GiftValidator
GiftRepository
ReceiptSender
GiftExporter
```

and one application service coordinating the use case.

31.84 Practical Exercise 2 — OCP

Current:

C#

```
switch (
    receiptType)
{
    case ReceiptType.Email:
        // ...
        break;

    case ReceiptType.Print:
        // ...
        break;
}
```

Refactor toward:

implementations.

Add a new sender without editing every existing receipt implementation.

31.85 Practical Exercise 3 — LSP

Interface:

```C#
public interface IGiftExporter
{
    Stream Export(
        Gift gift);
}
```

Bad implementation:

```C#
public Stream Export(
    Gift gift)
{
    throw new
        NotSupportedException();
}
```

Question:

Why suspicious?

Answer:

EXAMPLE / FLOW

```
The implementation claims the exporter role
while refusing its primary operation.
```

31.86 Practical Exercise 4 — ISP

Bad:

```C#
public interface IDataService
{
    Gift GetGift();
    void SaveGift();
    void SendReceipt();
    void Export();
    void ClearCache();
}
```

Split by cohesive capabilities.

31.87 Practical Exercise 5 — DIP

Bad:

```C#
public class GiftService
{
    private SqlGiftRepository
        _repository = new();
}
```

Better:

```C#
public GiftService(
    IGiftRepository repository)
```

31.88 Practical Exercise 6 — DRY

Find three duplicated validation rules.

Ask:

```EXAMPLE / FLOW
Same business knowledge?
```

If yes, centralize.

If merely visually similar, do not merge automatically.

31.89 Practical Exercise 7 — Encapsulation

Bad:

```C#
gift.Status = GiftStatus.Completed;
```

Better:

```C#
gift.Complete();
```

where `Complete()` checks valid transitions.

31.90 Practical Exercise 8 — Composition

Bad relationship:

```C#
class GiftService :
    EmailSender
```

Better:

```C#
class GiftService
{
    private readonly
        IEmailSender
        _emailSender;
}
```

31.91 Practical Exercise 9 — Explicit Dependencies

SOLID / DESIGN

Bad:

```
C#
GiftRepository.Current.Save(
    gift);
```

Better:

```
C#
public GiftService(
    IGiftRepository repository)
```

31.92 Practical Exercise 10 — KISS

Requirement:

```
EXAMPLE / FLOW
Only one receipt channel today
```

Start with:

```
EXAMPLE / FLOW
One simple implementation
```

Do not build a plugin system unless variation exists or is clearly required.

31.93 Practical Exercise 11 — Multi-Tenant Security

Bad repository API:

```
C#
FindAsync(
    giftId)
```

Ask whether tenant scoping is guaranteed elsewhere.

If not, a safer abstraction may require:

```
C#
FindAsync(
    tenantId,
    giftId,
    cancellationToken)
```

31.94 Practical Exercise 12 — Refactor Large Service

Given a service with:

```
EXAMPLE / FLOW
Validation
SQL
Email
PDF
HTTP payment
```

```
Audit
Cache
```

SOLID / DESIGN

Identify:

EXAMPLE / FLOW

```
Business responsibility
Application orchestration
Infrastructure dependencies
Cross-cutting concerns
```

before creating new classes.

Topic 31 — 25 Interview Questions

1. What is SOLID?

SOLID is a group of five object-oriented design principles:

EXAMPLE / FLOW

```
Single Responsibility
Open/Closed
Liskov Substitution
Interface Segregation
Dependency Inversion
```

They aim to reduce unnecessary coupling and improve maintainability, extensibility, and testability. Microsoft includes these five principles in its current .NET architecture guidance.

2. What is the Single Responsibility Principle?

SRP says a class/module should have one cohesive responsibility and one primary reason to change.

It does not mean one method per class.

A validator can have many validation methods while still having one cohesive responsibility.

3. How can constructor dependencies reveal an SRP problem?

A class requiring many unrelated dependencies may be coordinating too many responsibilities.

For example:

EXAMPLE / FLOW

```
Repository
SMTP
PDF
Payment
Cache
Audit
File system
Message broker
```

in one service should trigger a design review.

Microsoft explicitly notes this as an SRP smell in DI guidance.

4. What is the Open/Closed Principle?

OCP says software should be open to extension while stable existing code is relatively closed to modification.

For example, adding a new `IReceiptSender` implementation should ideally not require rewriting the application's existing receipt senders.

5. Does OCP mean existing code must never change?

No.

Bug fixes and requirement changes require modification.

The principle says known variation should be designed through extension points so that every new variant doesn't repeatedly require invasive modification of stable central logic.

6. What is the Liskov Substitution Principle?

LSP says a subtype or implementation must be safely substitutable for the abstraction/base type expected by callers.

It concerns behavioral contract, not only compiler-compatible inheritance.

7. What kinds of behavior matter for LSP?

Examples include:

EXAMPLE / FLOW

```
Accepted inputs
Returned outputs
State invariants
Expected side effects
Error semantics
Postconditions
```

An implementation that technically implements an interface but refuses valid contract operations can violate LSP.

8. Why is the Rectangle/Square example associated with LSP?

A mutable `Rectangle` contract may allow width and height to vary independently.

A `Square` requires them to remain equal.

Substituting Square can therefore violate behavior expected from Rectangle.

The problem is the contract, not whether inheritance syntax compiles.

9. What is Interface Segregation?

ISP says clients should not depend on operations they do not need.

Prefer focused role interfaces over large “manager” interfaces containing unrelated capabilities.

10. Does ISP require single-method interfaces?

No.

An interface can contain multiple methods when they form one cohesive role.

For example, a repository may reasonably expose several persistence operations.

11. What is Dependency Inversion?

DIP says high-level policy should depend on abstractions rather than low-level implementation details, and implementation details should conform to those abstractions.

12. What is the difference between DIP and dependency injection?

SOLID / DESIGN

DIP is a design principle.

DI is a technique used to supply dependencies to objects.

Constructor injection:

```
C#
public GiftService(
    IGiftRepository repository)
```

is one common mechanism for implementing a DIP-friendly design.

13. What is inversion of control?

IoC means control over creation or execution is handed to an external framework/container rather than the object controlling everything itself.

ASP.NET Core's DI container is an example.

14. Why is constructor injection usually preferred?

It makes required dependencies explicit.

A class cannot be constructed without the collaborators it needs, which improves:

EXAMPLE / FLOW

```
Discoverability
Testability
Validity
Transparency
```

Microsoft's DI guidance recommends constructor injection and avoiding direct instantiation of dependencies.

15. What is separation of concerns?

SoC means different kinds of work should reside in different components/layers.

Typical ASP.NET Core separation:

EXAMPLE / FLOW

```
Controller → HTTP
Application service → use case
Domain → business rules
Infrastructure → database/external systems
```

16. What is encapsulation?

Encapsulation protects internal state and exposes behavior through controlled APIs.

A `Gift` should ideally maintain its own invariants instead of exposing mutable state that any caller can modify arbitrarily.

17. What is DRY?

DRY means avoiding multiple independent implementations of the same knowledge/business rule.

It is not simply eliminating every visually duplicated line of code.

18. What is KISS?

KISS means keeping the design as simple as possible while correctly satisfying real requirements.

It protects against unnecessary factories, strategies, wrappers, abstractions, and layers.

19. What is YAGNI?

YAGNI means:

EXAMPLE / FLOW

```
You Aren't Gonna Need It
```

Do not build speculative features or extension mechanisms for hypothetical future requirements without evidence they are needed.

20. How do OCP and YAGNI coexist?

Use OCP for real or well-established variation.

Use YAGNI to avoid abstracting every hypothetical future possibility.

The balance is:

EXAMPLE / FLOW

```
Known variation → design extension point
Speculative variation → keep simple
```

21. Why is composition often preferred over inheritance?

Composition reduces coupling to a rigid hierarchy and allows behavior to be replaced independently.

Example:

EXAMPLE / FLOW

```
GiftService uses IReceiptSender
```

is often more flexible than:

EXAMPLE / FLOW

```
GiftService inherits ReceiptSender
```

when no genuine is-a relationship exists.

22. What is the Law of Demeter?

It encourages objects to collaborate mainly with direct dependencies rather than navigating deep internal object graphs.

Deep navigation:

C#

```
gift.Campaign.Owner.Department.Address.Country.Code
```

creates structural coupling.

23. Can SOLID hurt a project?

Misapplied SOLID can create overengineering:

EXAMPLE / FLOW

```
Too many interfaces
Too many factories
Too many tiny classes
Hard-to-follow object graphs
Speculative abstractions
```

Principles are decision tools, not mandatory templates.

24. How does SOLID improve testing?

SOLID commonly creates small components with explicit abstractions.

Instead of requiring:

EXAMPLE / FLOW

```
SQL Server
SMTP
HTTP gateway
```

a unit test can provide controlled implementations such as:

EXAMPLE / FLOW

```
Fake repository
Fake sender
Stub clock
```

This isolates the behavior being tested.

25. How would you apply SOLID to the Fundraising Platform?

A practical approach is:

EXAMPLE / FLOW

```
Gift
    →
owns Gift invariants

GiftApplicationService
    →
orchestrates Gift use cases

IGiftRepository
    →
persistence abstraction

GiftRepository
    →
EF Core implementation

IReceiptSender
    →
receipt capability

EmailReceiptSender
PrintReceiptSender
    →
replaceable implementations

IAuditWriter
    →
audit abstraction
```

```
Controller
    →
HTTP boundary
```

SOLID / DESIGN

Then use:

EXAMPLE / FLOW

```
SRP → cohesive responsibilities
OCP → add new receipt strategies
LSP → all implementations honor contracts
ISP → small capability interfaces
DIP → application depends on abstractions
```

Topic 31 — Quick Revision

Principle	Meaning
SRP	One cohesive responsibility/reason to change
OCP	Extend behavior without repeatedly modifying stable logic
LSP	Implementations preserve behavioral contract
ISP	Clients depend only on capabilities they need
DIP	Dependencies point toward abstractions
DI	Technique for supplying dependencies
IoC	Framework/container controls construction/execution
Cohesion	Related responsibilities belong together
Coupling	Degree of dependency between components
SoC	Separate different kinds of work
Encapsulation	Protect internal state/invariants
Information hiding	Hide implementation detail
DRY	One authority for one piece of knowledge
KISS	Prefer simplest adequate design
YAGNI	Do not build speculative requirements
Composition	Assemble behavior through collaborators
Inheritance	Use for genuine substitutable is-a relationships
Explicit Dependencies	Declare what a type needs
Law of Demeter	Avoid deep dependency navigation
Tell, Don't Ask	Put behavior near owned state

Topic 31 — Complete Fundraising Architecture Example

A clean Gift creation flow might look like:

EXAMPLE / FLOW

SOLID / DESIGN

```
HTTP POST /gifts
        ↓
GiftController
        ↓
CreateGiftService
        ↓
Gift.Create(...)
        ↓
IGiftRepository
        ↓
GiftRepository / EF Core
        ↓
Commit
        ↓
Receipt / Audit / Outbox orchestration
```

Controller

C#

```
[ApiController]
[Route("api/gifts")]
public sealed class
    GiftController
    :
    ControllerBase
{
    private readonly
        CreateGiftService
        _service;

    public GiftController(
        CreateGiftService service)
    {
        _service = service;
    }

    [HttpPost]
    public async Task<
        ActionResult<CreateGiftResponse>>
        Create(
            CreateGiftRequest request,
            CancellationToken cancellationToken)
    {
        CreateGiftResponse response =
            await _service.CreateAsync(
                request,
                cancellationToken);

        return CreatedAtAction(
            nameof(Create),
            new
            {
                giftId =
                    response.GiftId
            },
            response);
    }
}
```

Controller owns:

Domain Gift

C#

```
public sealed class Gift
{
    private readonly
        List<GiftAllocation>
        _allocations = [];

    private Gift(
        long tenantId,
        long constituentId,
        decimal amount)
    {
        if (
            tenantId <= 0)
        {
            throw new
                ArgumentOutOfRangeException(
                    nameof(tenantId));
        }

        if (
            constituentId <= 0)
        {
            throw new
                ArgumentOutOfRangeException(
                    nameof(constituentId));
        }

        if (
            amount <= 0m)
        {
            throw new
                ArgumentOutOfRangeException(
                    nameof(amount));
        }

        TenantId = tenantId;

        ConstituentId = constituentId;

        Amount = amount;

        Status = GiftStatus.Pending;
    }

    public long GiftId
    {
        get;
        private set;
    }

    public long TenantId
    {
        get;
    }

    public long ConstituentId
    {
```

```
        get;
    }

    public decimal Amount
    {
        get;
    }

    public GiftStatus Status
    {
        get;
        private set;
    }

    public IReadOnlyList<
        GiftAllocation>
        Allocations
        =>
        _allocations;

    public static Gift Create(
        long tenantId,
        long constituentId,
        decimal amount)
        =>
        new(
            tenantId,
            constituentId,
            amount);

    public void Complete()
    {
        if (
            Status
            !=
            GiftStatus.Pending)
        {
            throw new
                InvalidOperationException(
                    "Only pending gifts can be completed.");
        }

        Status = GiftStatus.Completed;
    }
}
```

Gift owns:

EXAMPLE / FLOW

```
Gift invariants
Gift state transition
```

Repository Abstraction

C#

```
public interface IGiftRepository
{
    Task AddAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Application Service

SOLID / DESIGN

C#

```
public sealed class
    CreateGiftService
{
    private readonly
        IGiftRepository
        _repository;

    private readonly
        IClock
        _clock;

    public CreateGiftService(
        IGiftRepository repository,
        IClock clock)
    {
        _repository = repository;

        _clock = clock;
    }

    public async Task<
        CreateGiftResponse>
        CreateAsync(
            CreateGiftRequest request,
            CancellationToken cancellationToken)
    {
        Gift gift =
            Gift.Create(
                request.TenantId,
                request.ConstituentId,
                request.Amount);

        await _repository.AddAsync(
            gift,
            cancellationToken);

        return new
            CreateGiftResponse(
                gift.GiftId,
                gift.Amount,
                gift.Status,
                _clock.UtcNow);
    }
}
```

Application service owns:

EXAMPLE / FLOW

Create Gift use-case orchestration

Infrastructure Repository

C#

```
public sealed class GiftRepository
    :
    IGiftRepository
{
    private readonly
        FundraisingDbContext
        _dbContext;
```

```
    public GiftRepository(
        FundraisingDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public async Task AddAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        await _dbContext.Gifts
            .AddAsync(
                gift,
                cancellationToken);

        await _dbContext
            .SaveChangesAsync(
                cancellationToken);
    }
}
```

Repository implementation owns:

```
EXAMPLE / FLOW
EF Core persistence details
```

Dependency Registration

```
C#
builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();

builder.Services
    .AddScoped<
        CreateGiftService>();
```

Final Architecture Flow

```
EXAMPLE / FLOW
Presentation
    ↓
HTTP Controller
    ↓
Application
    ↓
Use-Case Service
    ↓
Domain
    ↓
Gift / GiftAllocation
```

Compile-time infrastructure direction:

```
EXAMPLE / FLOW
Application/Core
    ↑
Abstraction
```

```
    ↑
Infrastructure implementation
```

Runtime flow may be:

EXAMPLE / FLOW

```
Controller
    ↓
Application Service
    ↓
IGiftRepository
    ↓
GiftRepository
    ↓
SQL Server
```

That is dependency inversion:

EXAMPLE / FLOW

```
Runtime call goes toward infrastructure

Compile-time policy dependency
points toward abstraction
```

Topics 30 + 31 — Combined Mental Model

Topic 30 asks:

EXAMPLE / FLOW

```
How does the runtime execute
and manage my objects?
```

Topic 31 asks:

EXAMPLE / FLOW

```
How should I organize those objects
so the application remains maintainable?
```

Together:

EXAMPLE / FLOW

```
C# Code
    ↓
Well-designed objects/services
    ↓
SOLID + cohesion + encapsulation
    ↓
CLR executes code
    ↓
Objects allocated
    ↓
Managed heap
    ↓
Application references determine lifetime
    ↓
GC reclaims unreachable objects
```

A poorly designed architecture can create poor memory behavior:

EXAMPLE / FLOW

```
Global singleton
    ↓
Huge cache
    ↓
Retains domain graph
    ↓
Objects promoted to Gen 2
    ↓
Memory grows
```

A better design might use:

EXAMPLE / FLOW

```
Explicit lifetime
Bounded cache
Small request-scoped services
Immutable/read-only projections
Streaming
Correct disposal
```

So software design and runtime behavior are connected.

Final Revision Flow

EXAMPLE / FLOW

```
Write business code
    ↓
Use SOLID to organize responsibilities
    ↓
Use DI to compose dependencies
    ↓
Use encapsulation to protect state
    ↓
Avoid unnecessary global state
    ↓
CLR executes methods
    ↓
Managed objects allocated
    ↓
Keep lifetimes appropriate
    ↓
GC handles unreachable objects
    ↓
Measure allocation/memory in production
    ↓
Optimize only proven hot paths
```

The two key lessons are:

> **Topic 30:** Garbage collection frees unreachable managed objects; it cannot fix objects that your architecture accidentally keeps reachable. Good memory behavior therefore depends heavily on lifetime, allocation, caching, pooling, and reference design.

Topic 31: SOLID is not about maximizing the number of interfaces and classes. It is about creating cohesive responsibilities, stable contracts, sensible extension points, substitutable implementations, focused interfaces, and dependency direction that keeps business policy independent of infrastructure details.

SOLID / DESIGN

Next roadmap topic: Topic 32 — Dependency Injection and Service Lifetimes.